

LLM für den Hausgebrauch – Large Language Models verständlich erklärt

Von Tokenisierung und Embeddings bis Transformer, Self-Attention und RAG

Karsten Keßler

2. August 2026

Inhaltsverzeichnis

Zusammenfassung	1
1 LLM für den Hausgebrauch	1
1.1 Motivation: Vom Token zur Antwort	4
1.2 Transformer	7
1.3 Mathematische Grundlagen	10
1.4 Ein Rechenbeispiel	17
1.5 Systeme um das Modell: Retrieval & Kontext (RAG)	31
1.6 Grenzen und Fallstricke von RAG	34
1.7 Systemübersicht	37
1.8 Modellarchitektur	37
1.9 Ausblick: Titans	44
1.10 Evaluation: Antworten prüfen	45
1.11 Grenzen des klassischen Language Modells	48
1.12 Fazit und Diskussion	49
1.13 Glossar	49
1.14 Literaturverzeichnis	51

Zusammenfassung

Dieses Dokument bietet eine technische Einführung in Large Language Models (LLMs) für Studierende und Praktiker:innen mit Programmiererfahrung. Im Fokus steht das Verständnis der zugrundeliegenden Mechanismen - von der Tokenisierung über Self-Attention bis zur Backpropagation.

Anders als produktfokussierte Tutorials behandelt dieses Material die mathematischen Grundlagen auf einem Niveau, das Intuition und Rechenfähigkeit verbindet. Begleitende Jupyter-Notebooks ermöglichen das praktische Nachvollziehen aller Konzepte.

Umfang: 50+ Seiten | ca. 12 interaktive Notebooks | Glossar mit 40+ Begriffen

Lizenz: CC BY-NC-SA 4.0

1 LLM für den Hausgebrauch

Inhaltlich führt dieses Dokument **Schritt für Schritt** durch die wichtigsten Bausteine moderner Large Language Models (LLMs): von der historischen Entwicklung der KI (Symbolic AI → Machine Lear-

ning → Deep Learning → LLMs) über das Trainingsprinzip **Self-Supervised Learning** (Next-Token-Prediction) bis zur grundlegenden Verarbeitungskette im Modell (**Tokenisierung** → **Embeddings** → **Transformer/Self-Attention** → **Logits/Softmax** → **Loss** → **Backpropagation**). Ein Schwerpunkt liegt auf der Intuition hinter **Self-Attention** (Query/Key/Value, Attention-Gewichte, Kontextvektoren) und darauf, wie daraus kontextabhängige Bedeutungen entstehen.

Darüber hinaus behandelt die Präsentation wichtige Systemaspekte jenseits des reinen Modells: **Vektor-Datenbanken** als semantischer Wissensindex, **Retrieval Augmented Generation (RAG)** als Kombination aus Retrieval und Generierung sowie (als Ausblick) **Memory-Konzepte** wie bei Titans. Abschließend werden typische Grenzen und Risiken von LLMs eingeordnet, u.a. **Halluzinationen**, begrenzte Kontextfenster, Kosten/Compute und Aspekte wie Robustheit und Sicherheit.

Zielgruppe: Das Material richtet sich an Studierende sowie technisch interessierte Praktiker:innen, die LLMs nicht nur „benutzen“, sondern die zentralen Konzepte und den Rechenweg dahinter verstehen möchten (Produkt- oder Toolkenntnisse sind nicht erforderlich).

Voraussetzungen: Erwartet werden grundlegende Programmiererfahrung (z. B. in Python) und ein Basisverständnis von Mathematik für Machine Learning (Vektoren/Matrizen, einfache Funktionen/Gradienten). Details zu Ableitungen, Softmax/Cross-Entropy oder Rechenbeispielen werden im Dokument schrittweise aufgebaut; tiefe Vorkenntnisse in Deep Learning oder Transformer-Architekturen sind nicht notwendig.

Wo es passt, sind zusätzlich **Links zu Python-Notebooks** angegeben, mit denen sich zentrale Konzepte praktisch nachvollziehen lassen (z. B. mathematische Grundlagen, Tokenisierung, Attention-Rechenbeispiele, Next-Token-Prediction oder semantische Suche). Dadurch eignet sich das Dokument sowohl als Begleitmaterial zur Präsentation als auch als Nachschlagewerk zum Wiederholen und Vertiefen.

Agenda

- 1 Motivation
- 2 Transformer-Architektur & Self-Attention
- 3 Mathematische Grundlagen
- 4 Ein Rechenbeispiel: Wie arbeitet das Modell?
- 5 Wie lernt das Modell?
- 6 Wie entscheidet das Modell?
- 7 Modellarchitekturen im Überblick
- 8 Titans: Ein Ausblick auf neue Architekturen
- 9 Grenzen von LLMs
- 10 Fazit und Diskussion
- 11 Glossar

Agenda und inhaltlicher Überblick des Dokuments

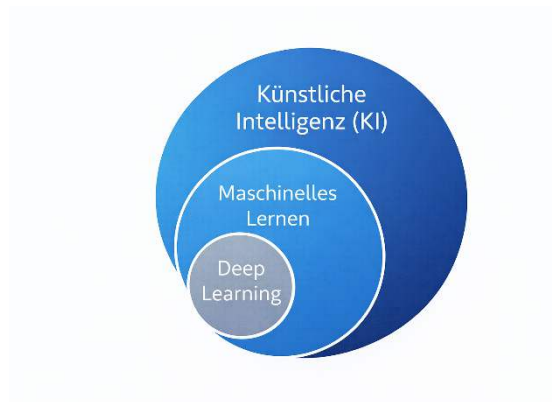
Einordnung

- Überblick über den roten Faden: Von Motivation und Begriffen bis zu Grenzen und Ausblick.
- Erwartungsmanagement: Fokus auf Grundprinzipien (Transformer, Training, Tokenisierung), nicht auf Produkt-marketing.
- Orientierung für die folgenden Kapitel und Demos: Welche Folien sind konzeptionell, welche rechnerisch.

1.1 Motivation: Vom Token zur Antwort

Wie Large Language Models wirklich funktionieren am Beispiel von:

„Paris ist die Hauptstadt von ...?“



Einordnung von KI, Machine Learning und Deep Learning

Künstliche Intelligenz → Maschinelles Lernen → Deep Learning

Maschinelles Lernen (ML) ist ein Teilgebiet der Künstlichen Intelligenz (KI), bei dem Computer aus Beispielen lernen, ohne explizit programmiert zu sein.

Ziel: - Muster erkennen - Vorhersagen treffen (z. B. Spamfilter, Bilderkennung)

Deep Learning (DL) ist eine spezielle Form des maschinellen Lernens, die auf tiefen künstlichen neuronalen Netzen mit vielen Schichten basiert.

Besonders geeignet für:

- Spracherkennung
- Bildanalyse
- Textgenerierung (z. B. GPT)

Generative Pretrained Transformer (GPT) ist ein *Deep-Learning-Modell*:

- **Generative:** erzeugt neue Texte (nicht nur Klassifikation)
- **Pretrained:** vortrainiert auf riesigen Textmengen
- **Transformer:** Architektur mit *Self-Attention*
→ Kontext über viele Tokens hinweg

Zentrale Fragen:

- Wie kann ein Modell ohne echtes Weltwissen korrekte Antworten erzeugen?
- Wie entsteht scheinbares „Verstehen“ allein durch Statistik?

Mensch	GPT
Kennt Bedeutung & Kontext	Kennt nur Wahrscheinlichkeiten
Kann abstrahieren & hinterfragen	Berechnet nächstwahrscheinliches Token
Lernt durch Erfahrung & Feedback	Lernt durch Milliarden Tokens & Loss

1.1.1 Live-Demo: Sprachmodell in Aktion (LM Studio)



Live-Demo einer Next-Token-Vorhersage aus Kontextvektor – Logits, Softmax, Vorhersage in LM Studio

Das Modell ergänzt Token für Token auf Basis statistischer Wahrscheinlichkeiten.

Einordnung

- Die Live-Demo illustriert dieses Prinzip konkret: Das Modell ergänzt Token rein statistisch ohne explizites Faktenwissen.
- Historische Entwicklung der KI: Symbolic AI → Machine Learning → Deep Learning → LLMs.
- Verschiebung von handcodierten Regeln zu datengetriebenen Repräsentationen (Features/Embeddings).
- Einordnung von GPT in die DL-/NLP-Landschaft: Transformer als Architektur, Decoder-only als Modellfamilie.

Einordnung von GPT & Co.

Das Modell sieht Texteingaben („Paris ist die Hauptstadt von“) und soll das nächste Token korrekt vorhersagen (z. B. „Frankreich“). Das Modell sieht dabei nur Tokens, keine Bedeutungen: „von“ schaut stark auf „Hauptstadt“ und etwas auf „Paris“, daraus entstehen die Gewichtungen.

Lernarten im Überblick

Lernart	Kurzbeschreibung	Typisches Beispiel
Supervised Learning	Lernen mit menschlichen Labels	Katze vs. Hund, Bildklassifikation
Self-Supervised Learning	Labels entstehen aus den Eingabedaten selbst	Next Token Prediction in GPT, Masked LM in BERT
Unsupervised Learning	Keine Labels, Modell erkennt Strukturen selbst	Clustering (z. B. K-Means), Word2Vec-Embeddings
Reinforcement Learning	Lernen über Belohnung/ Bestrafung nach Aktionen	Schach, Go, RLHF-Finetuning von ChatGPT

Einordnung von GPT in verschiedene Lernparadigmen

Warum hier Self-Supervised? Das Modell bekommt Milliarden Textbeispiele und „lernt aus sich selbst“: Aus dem Satz „Paris ist die Hauptstadt von Frankreich“ schneidet man z. B. „Frankreich“ ab – Aufgabe: Vorhersage dieses Wortes. Kein Mensch muss manuell annotieren, das macht Self-Supervised Lernen mächtig für LLMs. Technisch gesehen ist dieses Beispiel Supervised Learning (weil Eingabe und Ziel vorliegen), aber die Labels erzeugt der Text selbst, darum nennt man es Self-Supervised Learning.

Einordnung

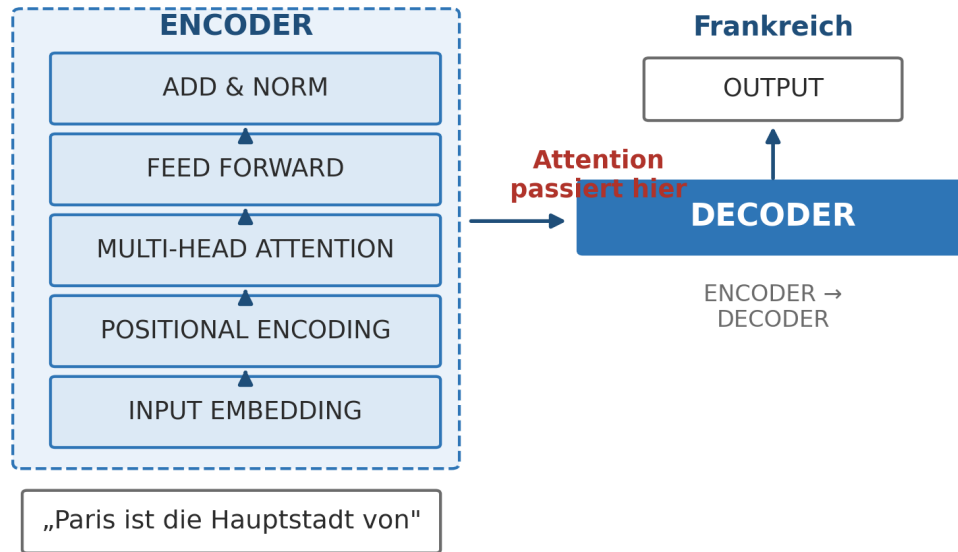
- Self-Supervised Learning: Next-Token-Prediction als Trainingsziel (Labels entstehen aus dem Text selbst).
- Abgrenzung zu Supervised, Unsupervised (Clustering/Embeddings) und RL/RLHF als Feintuning-Schritt.
- Intuition: Modelle lernen statistische Regularitäten und Weltwissen indirekt über Textzusammenhänge.
- Beispiel: Paris → Frankreich als Illustration für Kontextnutzung und semantische Assoziation.

1.2 Transformer

→ „Paris ist die Hauptstadt von“

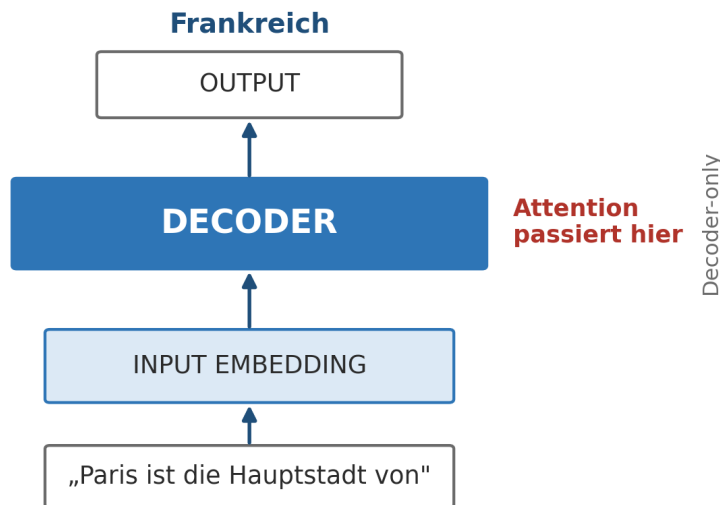
- **Tokenisierung:** Zerlegung in kleinere Tokens, z. B. „Paris“, „ist“, „die“ ...
- **Embedding:** Jedem Token einen Vektor zuweisen
- **Attention:** Kontext wird einberechnet – „Auf wen soll ich achten?“
- **Output:** Es wird ein Vektor für den nächsten Token erzeugt
- **Softmax:** Umwandlung in Wahrscheinlichkeiten für alle möglichen Wörter
- **Loss:** Vergleich mit dem Zielwort, um den Fehler zu berechnen
- **Backpropagation:** Fehler wird zurückgeleitet und die Gewichte angepasst, damit das Modell lernt

Decoder-Encoder-Architektur (z. B. T5, vereinfacht)



Decoder-Architektur (z. B. GPT)

Umgangssprachlich wird die Decoder-Architektur als "Language Modell" bezeichnet.



Architekturtyp	Beispiel	Anwendung	Merkmal
Encoder-only	BERT	Klassifikation, NER	Bidirektional (sieht ganze Eingabe)
Decoder-only	GPT	Textgenerierung	Autoregressiv (nur vorherige Tokens)
Encoder-Decoder	T5, BART, Marian	Übersetzung, Zusammenfassung	Cross-Attention

Einordnung

- Gesamtpipeline: Text $\rightarrow$ Tokenisierung $\rightarrow$ Embedding $\rightarrow$ (mehrfach) Self-Attention/FFN $\rightarrow$ Output-Logits $\rightarrow$ Softmax $\rightarrow$ Loss $\rightarrow$ Backpropagation.
- Trennung von Vorwärtsrechnung (Inference) und Training (zusätzlich Loss & Gradienten).
- Decoder-only (GPT) für Generierung, Encoder-only (BERT) für Verständnis/Maskierung, Encoder-Decoder (T5/BART) für Sequenz-zu-Sequenz.

1.3 Mathematische Grundlagen

Ziel: Verlustfunktion L minimieren.

Gradient = Richtungsvektor des stärksten Anstiegs:

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]$$

Kettenregel: $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial W}$

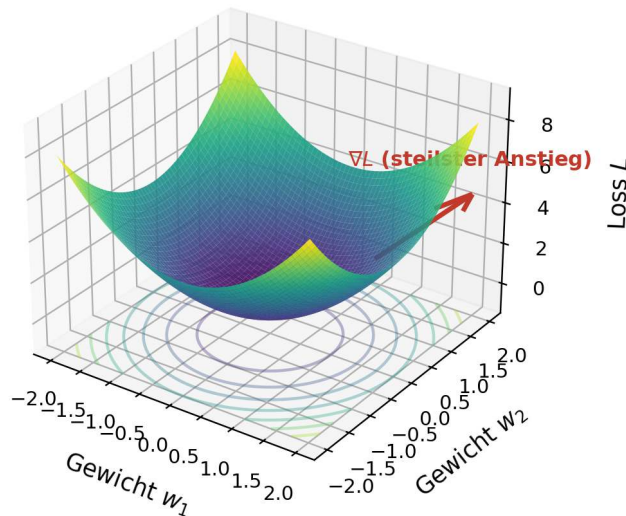
Beispiel für eine lineare Schicht: $z = W \cdot x \Rightarrow \frac{\partial z}{\partial W} = x^T \Rightarrow \frac{\partial L}{\partial W} = (p - y) \cdot x^T$

Update-Regel (Gradient Descent): $W_{\text{neu}} := W_{\text{alt}} - \eta \cdot \frac{\partial L}{\partial W}$

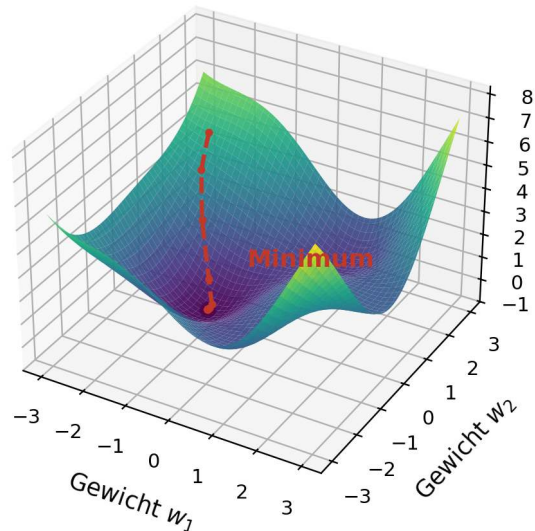
Was ist z ? Voraktivierung eines Neurons: $z = W \cdot x + b$ – eine lineare Kombination aus Gewichten und Eingabe. Die Größe, über die sich der Fehler an die Gewichte weitergibt.

Warum Kettenregel? Die Lossfunktion hängt nicht direkt von den Gewichten ab, sondern nur über den Umweg $W \rightarrow z \rightarrow L$. Darum gilt: $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial W}$

Loss-Landschaft (2 Gewichte)



Gradientenabstieg - Weg zum Minimum



Gradientenabstieg: Optimierung auf der Loss-Landschaft – Gradient, Kettenregel, Loss

Die Wörter erzeugen Vektoren. Diese Vektoren fließen in den Attention-Mechanismus, wo diese Wörter gegenseitig aufeinander „schauen“. Am Ende werden die Gewichte so optimiert, dass „von“ z. B. besser erkennt, dass „Hauptstadt“ wichtig ist. Dazu verwendet man die Kettenregel, weil der Fehler über mehrere Zwischenschritte von den Gewichtsmatrizen abhängt. Die Kettenregel erlaubt es, diese Zwischenschritte sauber zu verrechnen. Der Gradient sagt dem Modell, in welcher Richtung und wie stark die Gewichte geändert werden sollen, damit der Fehler verringert wird.

Einordnung

- Optimierung neuronaler Netze: Kettenregel, Gradient Descent, Interpretation der Loss-Landschaft.
- Rolle der Lernrate: Stabilität vs. Geschwindigkeit (Overshooting vs. langsame Konvergenz).
- Lokale Minima, Sattelpunkte und warum große Netze oft trotzdem gut trainierbar sind (Intuition).

Notebooks

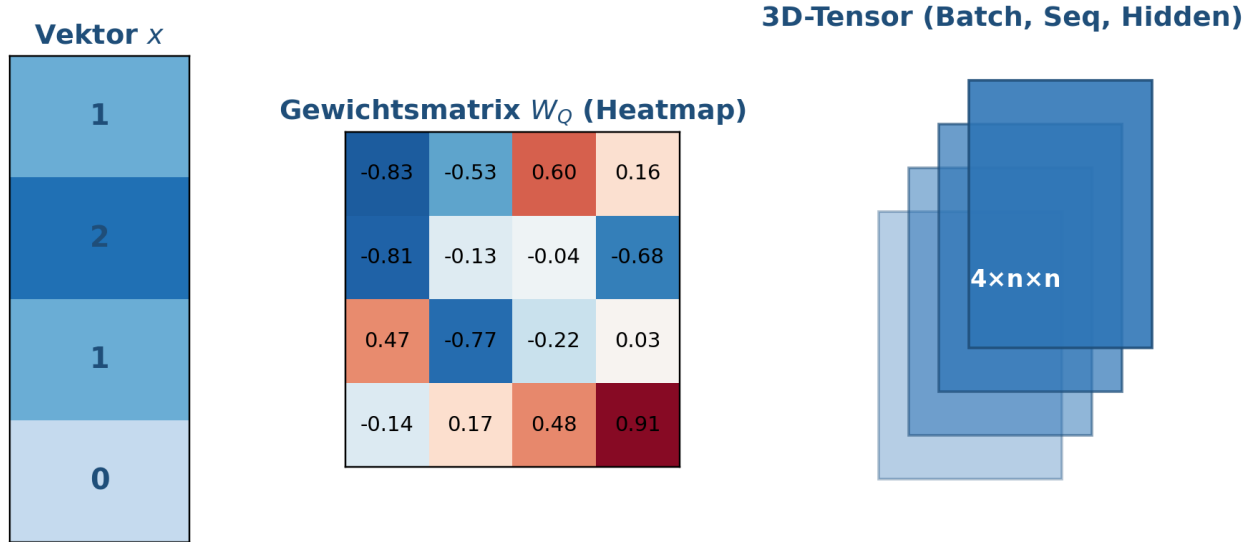
- Mathematische Grundlagen (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/kapitel_3_mathematische_grundlagen.ipynb): zeigt Ableitungen, Gradientenabstieg und Optimierung intuition an kleinen Beispielen.

1.3.1 Matrixrechnung

Matrixprodukt: $z = W \cdot x$ (W Gewichtsmatrix, x Eingabevektor, z. B. $x = (1, 2, 1, 0)$, z Vektor mit anders codiertem Inhalt). $\frac{\partial(Wx)}{\partial W} = x^T$, also $\frac{\partial L}{\partial W} = (p - y) \cdot x^T$ (äußeres Produkt) – interpretierbar als Gradientenmatrix.

Dimensionsregeln: $W \in \mathbb{R}^{n \times d}$, $x \in \mathbb{R}^d$, $z \in \mathbb{R}^n$.

Matrizenrechnung in neuronalen Netzen bedeutet: Man mischt und gewichtet die alten Informationen neu, um passendere Bedeutungsvektoren z zu erzeugen. Die Eingabevektoren für „Paris“, „Hauptstadt“, „von“ sind bereits in diesem Vektorraum (tatsächlich z. B. im $\mathbb{R}^{768}$). Die Matrizen W_Q, W_K, W_V transformieren diesen Vektorraum von x nach z durch Projektion, Drehung usw. – Attention operiert auf den geometrischen Beziehungen in diesem Raum.



Batch (wie viele Beispiele gleichzeitig): Netz wird parallel gerechnet, z. B. 32, 64 oder 128 Beispiele. Sequenz: LLMs verarbeiten eine Sequenz von Tokens – GPT-2-Tensor $T \in \mathbb{R}^{32 \times 512 \times 768}$: 32 Sätze parallel, jeder Satz 512 Tokens, jeder Token ein 768-dimensionales Embedding.

Lineare Transformation: $z = W \cdot x$ als Basis neuronaler Netze – Matrix, Vektor, Projektion

Einordnung

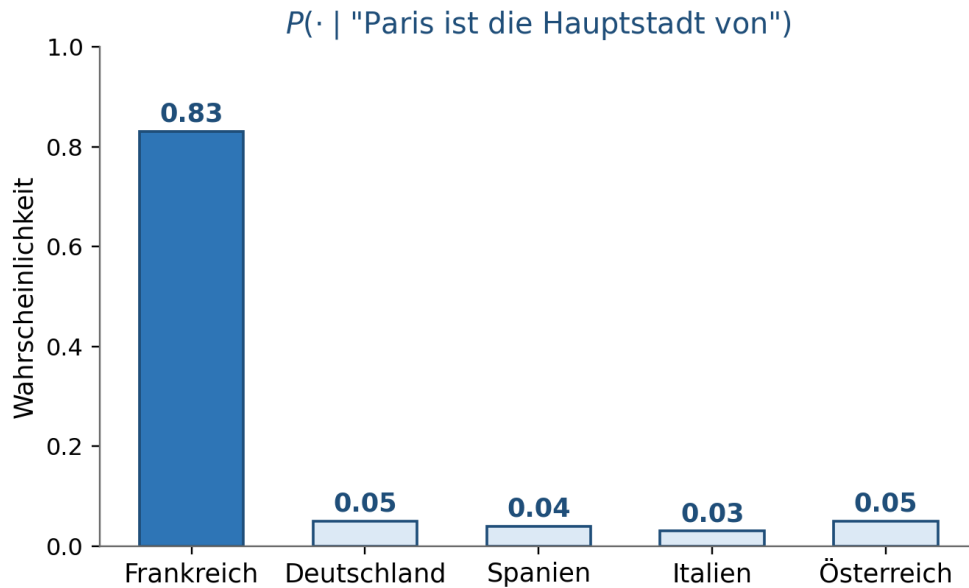
- Matrixrechnung als Grundbaustein: Lineare Projektionen transformieren Vektoren in neue Repräsentationsräume.
- $z = W \cdot x$: Dimensionalitäten erklären (Eingabevektor, Gewichtsmatrix, Ausgabedimension).
- Verbindung zu Transformer: W_Q, W_K, W_V und weitere Projektionsmatrizen sind genau solche linearen Abbildungen.
- Interpretation: Gewichte kodieren gelernte Muster; Multiplikation ist „Feature-Mischung“/Aggregation.

1.3.2 Wahrscheinlichkeitsrechnung

LLMs sind letztendlich Wahrscheinlichkeitsmodelle: Sie lernen, für jede mögliche Fortsetzung eines Textes die bedingte Wahrscheinlichkeit korrekt vorherzusagen.

- $P(E)$ – Wahrscheinlichkeit des Ereignisses E
- $P(A|B)$ – bedingte Wahrscheinlichkeit von A unter der Bedingung B
- **Satz von Bayes:** $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$
- **Erwartungswert:** $E[X] = \sum_i x_i \cdot P(x_i)$

Klassifikation in LLMs: Likelihood $P(y|x)$ maximieren, gerechnet mit Cross-Entropy-Loss. Für die Eingabe-Sequenz „Paris ist die Hauptstadt von“ berechnet das LLM $P(\text{Frankreich} | \text{Paris ist die Hauptstadt von})$. Cross-Entropy vergleicht diese Vorhersage mit dem tatsächlichen Zielwort.



Einordnung

- Von Logits zu Softmax: Das Modell liefert zunächst unnormierte Scores (Logits) pro Token.
- Softmax macht daraus eine Wahrscheinlichkeitsverteilung über das Vokabular: $p(\text{next token} | \text{Kontext})$.
- Numerische Stabilität: Softmax wird praktisch stabil gerechnet (z. B. LogSumExp-Trick).
- Vorbereitung für Training: Cross-Entropy vergleicht die Verteilung mit dem Ziel-Token.

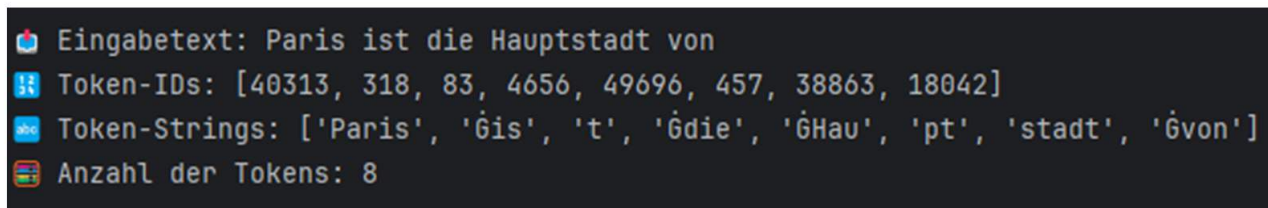
1.3.3 Tokenisierung

Beispiel-Wortfolge: „Paris“, „ist“, „die“, „Hauptstadt“, „von“ → Tokenfolge: Paris, Ġis, t, Ġdie, ĠHau, pt, stadt, Ġvon. Ein Wort besteht also aus mehreren Tokens.

Live-Demo in LM Studio: Eingabe „Paris ist die Hauptstadt von“, Modell `google/gemma-3-12b`, Ausgabe „Paris ist die Hauptstadt von Frankreich.“ (4.63 tok/sec, 8 Tokens).

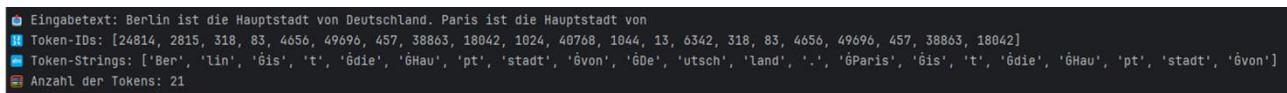


Terminal-Ausgabe (Tokenisierung im Detail):



Die Token-ID entspricht dem Index des Tokens im Vokabular. Das Vokabular ist groß (Zehntausende Tokens), aber dadurch kann das Modell praktisch jeden Text darstellen – und gegenüber der Gesamtzahl aller möglichen Wörter ist es sogar eher klein.

Zweites Beispiel („Berlin ist die Hauptstadt von Deutschland. Paris ist die Hauptstadt von“, 21 Tokens):



Embeddings als Matrix $X \in \mathbb{R}^{21 \times 768}$ (GPT-2). Jeder Token wird in einen Vektor der Länge 768 abgebildet.

Tokenisierung eines Satzes in Subword-Tokens – Tokenisierung, Tokens, Vokabular

Einordnung

- Sprache wird als Token-Sequenz modelliert (diskrete Einheiten statt „ganze Sätze“).
- Token können Wörter, Subwords oder Zeichen/Sonderzeichen sein (abhängig vom Tokenizer).
- Tokenisierung legt fest, welche Muster das Modell überhaupt unterscheiden/lernen kann (OOV, Kompression, Mehrdeutigkeit).
- Praktische Konsequenz: Prompt-Design und Kosten hängen stark an der Tokenanzahl.

Notebooks

- Tokenisierung (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/tokenisierung_beispiel.ipynb): demonstriert, wie Text in Tokens zerlegt wird und wie sich das auf Länge und Kosten auswirkt.

1.3.4 Embeddings

Ein Embedding ist ein Zahlenvektor, der die Bedeutung eines Tokens im Modell darstellt. Jeder Token wird in einen Vektor im Raum $\mathbb{R}^{768}$ umgewandelt. Der Token „Paris“ hat in „*Paris ist die Hauptstadt von*“ zunächst den gleichen Vektor wie in „*Paris hat gestern gewonnen*“ (Token-Embedding roh, trägt nur Grundinformationen):

Vektor (erste 10 Werte) = $(-0.229, 0.065, -0.211, -0.089, -0.02, -0.31, 0.286, 0.213, -0.179, 0.119)$

Kontextuelles Embedding: Dasselbe Token hat je nach Kontext unterschiedliche kontextualisierte Vektoren, weil der Vektor durch Attention verändert wird – ein Vergleich der ersten 10 Werte von rohem vs. kontextualisiertem Embedding zeigt das.

Die Embedding-Vektoren entstehen beim Training des Sprachmodells und sind fest im Modell gespeichert. Bei der Verwendung wird jeder Token in der gelernten Embedding-Matrix nachgeschlagen – kein Training und keine Veränderung mehr. Mathematisches Arbeiten mit Bedeutungsräumen: Wörter werden in Vektoren übersetzt, Vektoren können per Kosinus-Ähnlichkeit verglichen werden.

Einordnung

- Embeddings sind gelernte, feste Vektoren pro Token – Nachschlagen in der Embedding-Matrix, kein Training zur Laufzeit.
- Roh-Embedding ist zunächst kontextunabhängig: dasselbe Token startet überall mit demselben Vektor.
- Erst Self-Attention macht daraus kontextabhängige Bedeutung (siehe folgende Folien).
- Ähnlichkeit zwischen Vektoren lässt sich über Kosinus-Ähnlichkeit im Embedding-Raum messen.

1.4 Ein Rechenbeispiel

Wie arbeitet das Modell? Betrachtet wird ein konkretes Mini-Beispiel, in dem das Token „von“ im Fokus steht. Ziel: Das Modell soll lernen, dass das Wort „von“ in diesem Satz besonders eng mit „Hauptstadt“ und „Frankreich“ verknüpft ist.

Beispiel-Satz: „*Paris ist die Hauptstadt von Frankreich.*“ Fokus: Das Wort „von“ steht im Satz zwischen „Hauptstadt“ und „Frankreich“.

Dafür berechnet das Modell:

1. Einen Query-Vektor Q für „von“.
2. Key- und Value-Vektoren K/V für die anderen Wörter im Satz.
3. Attention-Scores: Wer ist wichtig für „von“? Jeder Token betrachtet alle anderen Tokens, inklusive sich selbst – das nennt man Self-Attention.
4. Einen Output-Vektor, also den kontextualisierten Vektor für „von“.

$Q/K/V$ entstehen aus Gewichtsmatrizen. „von“ vergleicht sich mit allen anderen. Bei Fehlern werden die Gewichte mit Backpropagation angepasst.

Einordnung

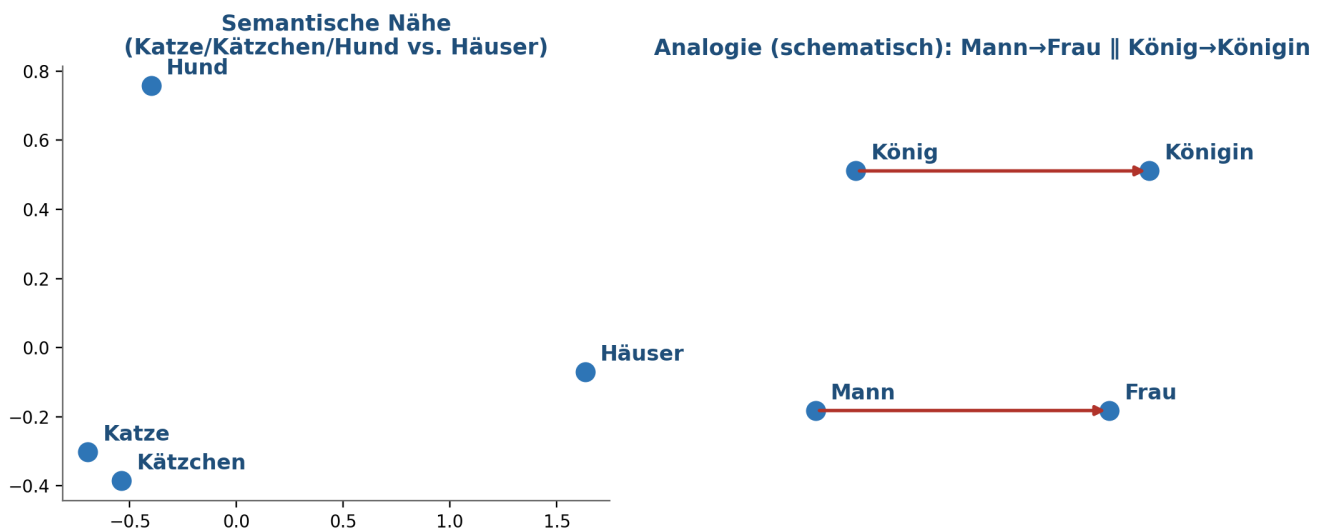
- Ausgangspunkt: Tokens sind bereits in Embeddings überführt; das Modell verarbeitet alle Tokens parallel.
- Self-Attention berechnet für jedes Token eine gewichtete Mischung der anderen Tokens (Kontextaggregation).
- Ergebnis: Kontextabhängige Bedeutung (z. B. „von“ als Relation, nicht isoliertes Wort).
- Intuition: Statt fester Regeln werden Relevanzen aus Daten gelernt (Gewichtungen als „weiche Regeln“).

1.4.1 Wort-Embeddings visualisieren

Wort-Embeddings kodieren semantische Bedeutung in Vektoren – ähnliche Bedeutungen ergeben ähnliche Vektoren. Das folgende Beispiel nutzt zur Veranschaulichung 7 semantische Dimensionen (tatsächlich sind es hunderte oder tausende):

Wort	Lebewesen	Katzenartig	Menschlich	Geschlecht	Königlich	Verb	Plural
Katze	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2
Kätzchen	0.5	0.8	-0.1	0.2	-0.6	-0.5	-0.1
Hund	0.7	-0.1	0.4	0.3	-0.4	-0.1	-0.3
Häuser	-0.8	-0.4	-0.5	0.1	-0.9	0.3	0.8
Mann	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
Frau	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
König	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
Königin	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Wort-Embeddings visualisiert (PCA-Projektion auf 2D)



Wort-Embeddings und semantische Ähnlichkeit im Vektorraum

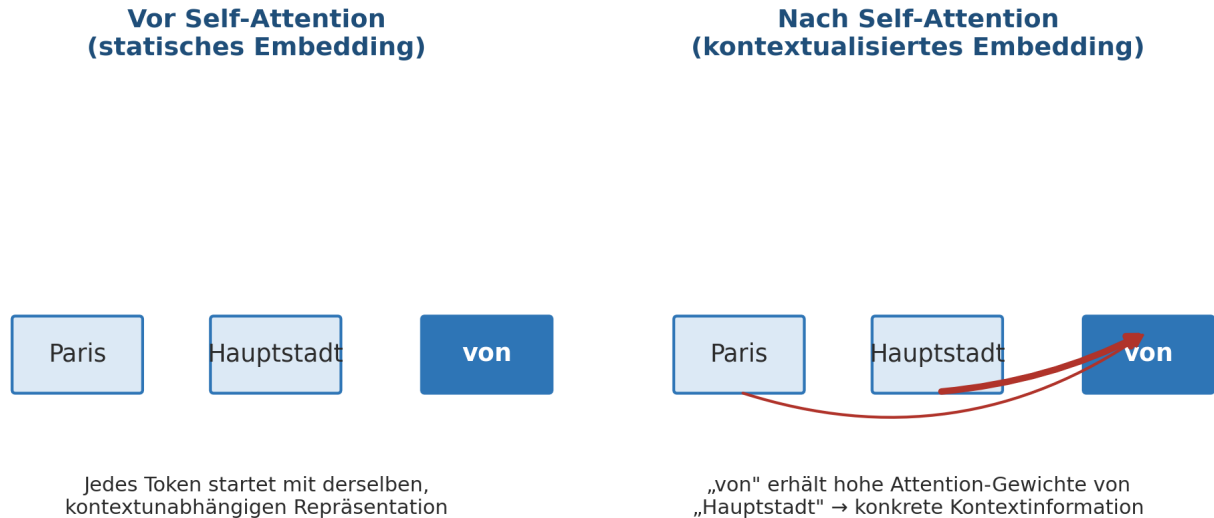
Einordnung

- Wort-Embeddings kodieren Bedeutung in kontinuierlichen Vektorräumen (Distributional Semantics).
- Nähe im Raum korreliert mit semantischer Ähnlichkeit; Richtungen können Relationen ausdrücken (Analogien).
- Embeddings sind die Schnittstelle zwischen diskreten Tokens und differentiellen neuronalen Netzen.

1.4.2 Kontextualisierte Embeddings vor und nach Self-Attention

Vor Self-Attention (statisches Embedding): Jedes Token besitzt ein kontextunabhängiges Embedding. Der Vektor repräsentiert die allgemeine Bedeutung von „von“ – alle Vorkommen starten mit ähnlicher Repräsentation (grammatische Präposition, allgemeine Relation).

Nach Self-Attention (kontextualisiertes Embedding): Das Embedding von „von“ wird unter Berücksichtigung aller anderen Tokens neu berechnet. Tokens wie „Hauptstadt“ erhalten hohe Attention-Gewichte, der neue Vektor enthält konkrete Kontextinformation (Beziehung Hauptstadt–Staat).



Wort-Embeddings sind im Transformer nicht statisch. Durch Self-Attention wird jedes Token-Embedding kontextabhängig angepasst. **Merksatz:** Self-Attention verwandelt statische Wort-Embeddings in kontextualisierte Bedeutungen.

Self-Attention: Kontextabhängige Bedeutungen – Attention, QKV, Kontext

Einordnung

- Self-Attention macht Embeddings kontextabhängig: gleiche Token-ID kann je nach Satz verschiedene Vektoren erhalten.
- Das reduziert Mehrdeutigkeit (Polysemie) und kodiert syntaktische/semantische Rollen.
- Beispiel: „von“ wird zur Relation zwischen „Hauptstadt“ und „Frankreich“ statt eigenständigem Inhalt.

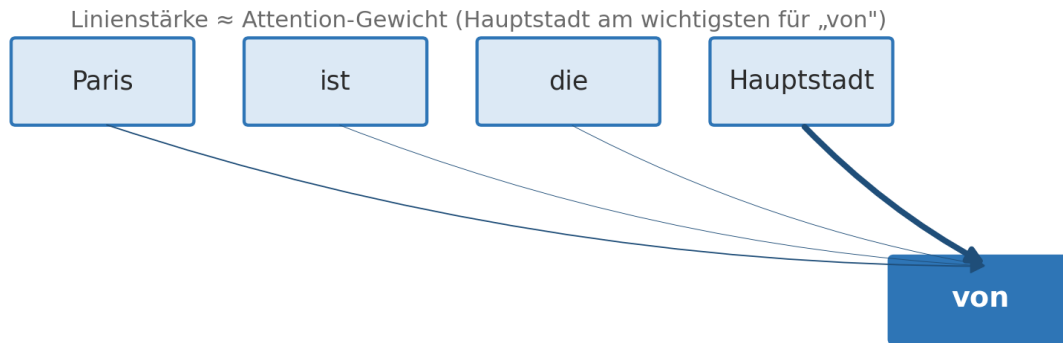
1.4.3 Self-Attention

Jeder Token wird durch Self-Attention abhängig vom Kontext neu berechnet. Vor Self-Attention wird Positions-Information codiert (Positional Encoding). Die Attention-Gewichte sagen, welche Vektoren (Wörter) wichtig sind – es entsteht ein neuer kontextualisierter Vektor pro Token.

Self-Attention-Schritt (Q, K, V und Gewichtung): „von“ bildet einen Query-Vektor $Q_{von} = W_Q \cdot x_{von}$, alle Wörter berechnen Key und Value: $K_j = W_K \cdot x_j$, $V_j = W_V \cdot x_j$. W_Q, W_K, W_V sind gelernte Gewichtsmatrizen in $\mathbb{R}^{d \times d}$ ($d = 64, 128, 768, \dots$). Multi-Head Attention berechnet mehrere Sets von Q, K, V gleichzeitig.

Vergleich: Wie ähnlich ist Q_{von} mit jedem K_j ? $Score_j = \frac{Q_{von} \cdot K_j^T}{\sqrt{d}}$ – ein Maß dafür, wie sehr j zu „von“ passt.

Self-Attention: Wer ist wichtig für „von“?



$$Q_{von} = W_Q \cdot x_{von} \quad K_j = W_K \cdot x_j \quad V_j = W_V \cdot x_j \quad Score_j = \frac{Q_{von} \cdot K_j^T}{\sqrt{d}}$$

Einordnung

- Q, K, V sind lineare Projektionen der Token-Embeddings und definieren die Attention-Berechnung.
- Query: „Worauf soll ich achten?“ Key: „Was biete ich an?“ Value: „Welche Information liefere ich?“
- Attention-Gewichte entstehen aus Query-Key-Ähnlichkeiten; Values werden entsprechend gemischt.
- Multi-Head Attention: mehrere parallele Sichtweisen auf Kontext (verschiedene Projektionen).

1.4.4 Attention-Berechnung Schritt für Schritt

Schritt 1–2 (Eingabevektoren und Gewichtsmatrizen): W_Q = Einheitsmatrix 4×4 , $W_K = W_V = 0.5 \cdot I$ (Vektoren zur Veranschaulichung frei gewählt).

Wort	Eingabevektor x	Key K_j	Value V_j
Paris	(2, 0, 1, 1)	(1.0, 0.0, 0.5, 0.5)	(1.0, 0.0, 0.5, 0.5)
ist	(0, 2, 0, 1)	(0.0, 1.0, 0.0, 0.5)	(0.0, 1.0, 0.0, 0.5)
die	(1, 1, 0, 0)	(0.5, 0.5, 0.0, 0.0)	(0.5, 0.5, 0.0, 0.0)
Hauptstadt	(0, 1, 3, 1)	(0.0, 0.5, 1.5, 0.5)	(0.0, 0.5, 1.5, 0.5)
von (Fokus)	(1, 2, 1, 0)	–	–

Schritt 3: $Q_{von} = W_Q \cdot x_{von} = (1, 2, 1, 0)$.

Schritt 5–6 (Scores mit $d = 4$, $\sqrt{d} = 2$, und Softmax-Gewichte):

Wort	Score	Softmax-Gewicht
Paris	0.75	0.203
ist	1.00	0.260
die	0.75	0.203
Hauptstadt	1.25	0.334

(Summe $e^{\text{Score}} = 10.45$)

Schritt 8 (Output-Vektor): gewichtete Summe der Value-Vektoren:

$$0.203 \cdot V_{Paris} + 0.260 \cdot V_{ist} + 0.203 \cdot V_{die} + 0.334 \cdot V_{Hauptstadt}$$

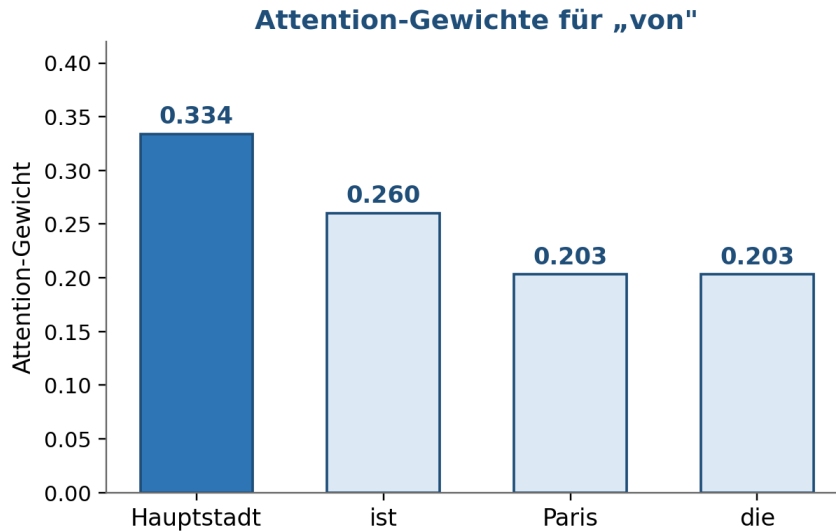
Auch W_K und W_V werden durch denselben Fehler (Loss) angepasst wie W_Q – alle drei Matrizen werden gleichzeitig trainiert.

Einordnung

- Score-Berechnung als skaliertes Skalarprodukt: $\text{Score}_{i,j} = \frac{Q_i \cdot K_j}{\sqrt{d_k}}$.
- Skalierung mit $\sqrt{d_k}$ stabilisiert Wertebereiche und Gradienten bei größeren Dimensionen.
- Softmax über Scores liefert Attention-Gewichte (Summe = 1), interpretierbar als Relevanzen.
- Kontextvektor entsteht als gewichtete Summe der Value-Vektoren.

1.4.5 Ergebnis der Attention

Kontextvektor: (0.304, 0.529, 0.603, 0.399). Dieser Vektor ist der neue kontextualisierte Bedeutungsvektor für das Token „von“ – er setzt sich aus den gewichteten Beiträgen der vorherigen Tokens zusammen. Ursprünglich $x_{von} = (1, 2, 1, 0)$, nach Attention (0.304, 0.529, 0.603, 0.399).



Architektur-Vergleich: Rechenbeispiel vs. GPT/BERT

	Rechenbeispiel	GPT/BERT
Vektorgroße	4	768 oder mehr
Attention-Heads	1	12 (BERT Base) bis 96 (GPT-3)
Schichten	1	12 (BERT Base) bis 96 (GPT-3)
Gewichtsmatrizen	$1 \times W_Q, W_K, W_V$	pro Head eigene Matrizen
Berechnung	einmalig	mehrschichtig mit Residual und LN

Residual = Skip-Verbindungen, LN = Layer Normalization zur Stabilisierung.

📖 Einordnung

- Ergebnis der Attention: neuer Vektor pro Token, der Bedeutung im Satzkontext kodiert.
- Damit kann das Modell Relationen (Subjekt/Objekt, „von“-Beziehungen, Referenzen) rechnerisch abbilden.
- Wichtig: „Verstehen“ im Transformer ist Ergebnis von Matrixoperationen, nicht symbolischer Regeln.

📓 Notebooks

- Attention-Rechnung (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/attention_rechenbeispiel.ipynb): rechnet Q/K/V, Scores, Softmax und Kontextvektor Schritt für Schritt durch.
- Multi-Head Attention Toy (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/multihead_attention_toy.ipynb): vergleicht zwei Heads mit unterschiedlichen Aufmerksamkeitsmustern auf denselben Satz.

1.4.6 Next Token Vorhersage

Wie entsteht der nächste Token? Ausgangspunkt ist der Kontextvektor nach Attention: $h = (0.304, 0.529, 0.603, 0.399)$.

1. **Lineare Projektion auf den Vokabelraum:** Die Gewichtsmatrix W_{out} transformiert den Vektor in einen Logit-Vektor (Scores für alle möglichen nächsten Tokens): $z = W_{out} \cdot h + b$. W_{out} hat die Dimension Vokabulargröße $\times$ Hidden-Size, also z. B. 50.000×768 in GPT.
2. **Berechnung der Logits (Rohwerte):** $z = (z_{Frankreich}, z_{Deutschland}, z_{Spanien}, \dots)$ – reelle Zahlen, z. B. $z_{Frankreich} = 3.2, z_{Deutschland} = 1.7$.
3. **Softmax über die Logits:** $p_{Token} = \frac{e^{z_{Token}}}{\sum_k e^{z_k}}$

Im Training vergleicht das Modell diese Wahrscheinlichkeiten mit dem tatsächlichen nächsten Wort. Daraus wird ein Fehler berechnet (Cross-Entropy-Loss). Dieser Fehler wird rückwärts durch das Netz propagiert, um die Matrizen W_{out}, W_Q, W_K, W_V zu verbessern.

Einordnung

- Next-Token-Prediction: Aus dem Kontextvektor wird über eine lineare Projektion ein Logit pro Vokabel berechnet.
- Softmax macht daraus Wahrscheinlichkeiten; Sampling/Decoding bestimmt das tatsächlich generierte Token.
- Training treibt die richtigen Tokens hoch, indem der Loss (Cross-Entropy) minimiert wird.

1.4.7 Softmax-Berechnung aus Logits

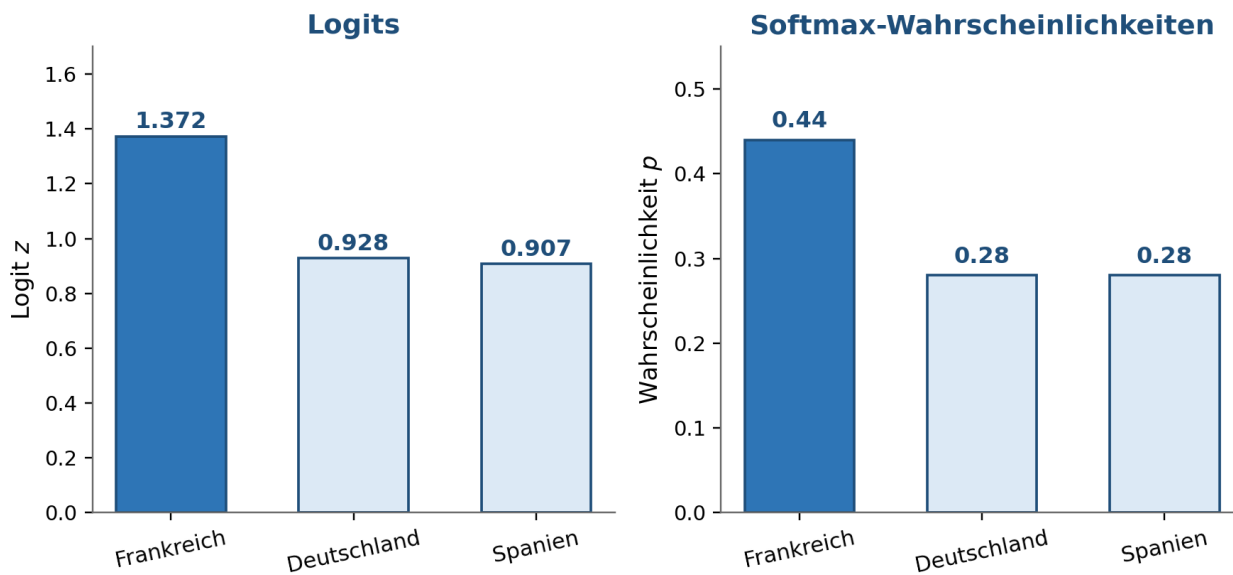
Schritt 4 (beispielhafte Illustration):

	Logit z	e^z	Wahrscheinlichkeit
Frankreich	3.2	24.53	0.74
Deutschland	1.7	5.47	0.16
Spanien	1.2	3.32	0.10

Ergebnis: Das Modell wählt mit höchster Wahrscheinlichkeit den nächsten Token „Frankreich“. Pipeline: Attention-Output $h \rightarrow W_{out} \rightarrow$ Logits $\rightarrow$ Softmax $\rightarrow$ Wahrscheinlichkeiten.

Schritt 5 (konkrete Berechnung): Voraussetzungen $h = (0.304, 0.529, 0.603, 0.399)$, Vokabular = (Frankreich, Deutschland, Spanien). Gewichtungsvektoren (Zeile in W_{out}): Frankreich $(1.0, 0.5, 1.0, 0.5)$, Deutschland $(0.0, 1.0, 0.0, 1.0)$, Spanien $(1.0, 0.0, 1.0, 0.0)$.

Logits: $z_{Frankreich} = 1.372$, $z_{Deutschland} = 0.928$, $z_{Spanien} = 0.907$. Exponentialwerte: $e^{1.372} = 3.94$, $e^{0.928} = 2.53$, $e^{0.907} = 2.48$ (Summe = 8.95).



Wahrscheinlichkeiten: $p_{Frankreich} = 3.94/8.95 = 0.44$, $p_{Deutschland} = 2.53/8.95 = 0.28$, $p_{Spanien} = 2.48/8.95 = 0.28$. Frankreich hat mit Logit 1.372 die höchste Softmax-Wahrscheinlichkeit (44 %).

Einordnung

- Mini-Vokabular macht Softmax konkret: Logits werden exponentiert und normiert.
- Kleine Logit-Änderungen können Wahrscheinlichkeiten stark verändern (Sensitivity).
- Decoding-Strategien (Greedy, Temperature, Top-k/Top-p) steuern Vielfalt und Determinismus.
- Training optimiert die Wahrscheinlichkeiten (Loss-Minimierung); Decoding entscheidet, wie daraus Text entsteht.

Notebooks

- Next Token Vorhersage 1 (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/next_token.ipynb): zeigt den Weg von Logits zu Wahrscheinlichkeiten und Token-Auswahl.
- Next Token Vorhersage 2 (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/next_token_prediction_toy.ipynb): vertieft Decoding mit einem kleinen, voll nachvollziehbaren Toy-Vokabular.
- Causal Masking Rechenbeispiel (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/causal_masking_rechenbeispiel.ipynb): zeigt, wie Decoder-Attention Zukunfts-Token per Maske ausschliesst.

1.4.8 Kontextvektor und Weiterverarbeitung

Nach dem Lernprozess (z. B. Schritt 3–5) wird der gelernte Kontextvektor weiterverarbeitet, z. B. durch Feedforward-Netze und LayerNorm. Diese Schritte wurden im Lernbeispiel zur Vereinfachung weggelassen. Die Zwischenschritte LayerNorm, Feedforward etc. dienen dazu, das Modell robuster und leistungsfähiger zu machen, sind aber für das einfache Lernbeispiel nicht entscheidend – hier betrachten wir nur den Self-Attention-Schritt.

In GPT/BERT folgen danach noch weitere Verarbeitungsschritte wie Feedforward-Blöcke (MLP), Residual Connections und Layer Normalization, die hier zur Vereinfachung ausgelassen wurden.

Einordnung

- Weitere Aspekte des Lernprozesses: Regularisierung, Overfitting, Trainingsdatenqualität.
- Praktische Stellschrauben: Batchgröße, Learning-Rate-Schedule, Weight Decay.
- Intuition: Gute Performance entsteht aus Daten, Architektur und Training zusammen (nicht nur „mehr Parameter“).

Notebooks

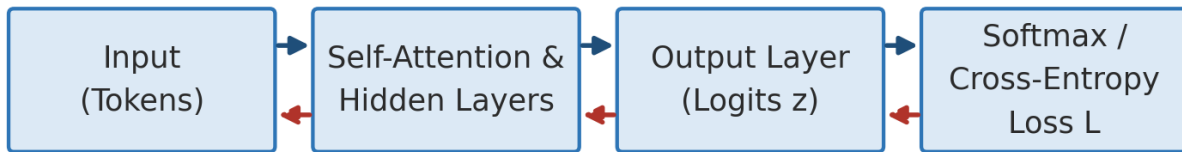
- Gradientenabstieg Iteration (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/gradient_descent_wq_diagonal.ipynb): visualisiert, wie Gewichte iterativ entlang des Gradienten aktualisiert werden.

1.4.9 Wie lernt das Modell?

$$\frac{\partial L}{\partial W} = (p - y) \cdot x^T \quad W_{\text{neu}} = W_{\text{alt}} - \eta \cdot \frac{\partial L}{\partial W}$$

Backpropagation: Der berechnete Gradient wird durch das Netzwerk zurückgerechnet und aktualisiert die Gewichte entlang aller Schichten – Wiederholung über Millionen Datensätze.

Vorwärtsrichtung: Vorhersage erzeugen



Rückwärtsrichtung: Fehler rückwärts weiterleiten (Rückpropagation, Gradient)

📖 Einordnung

- Zusammenfassung: Optimierung minimiert Loss; Gradienten sagen, wie Parameter angepasst werden.
- Blick auf die Praxis: Pretraining vs. Finetuning und warum Datenmengen entscheidend sind.
- Übergang zu Architekturfragen: Was genau wird in welchen Schichten gelernt?

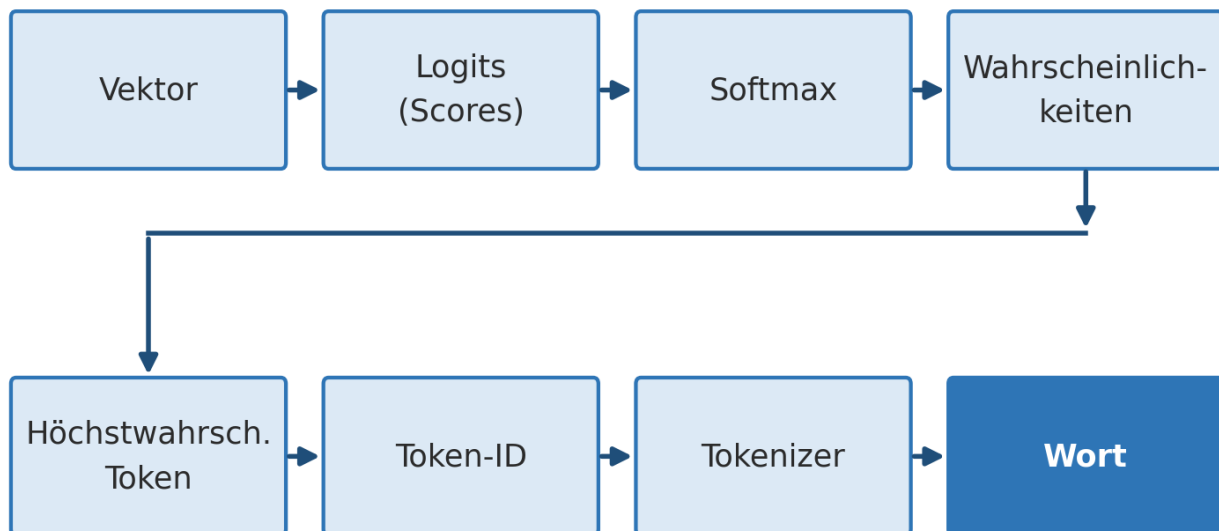
1.4.10 Wie entscheidet das Modell: Decoder-Vorhersage

Beim vorherigen Lernen von W_Q lernte das Modell, wie es aus dem Eingabekontext eine sinnvolle Repräsentation erzeugt (z). Jetzt lernt es, wie es diese Repräsentation verwendet, um den richtigen nächsten Token vorherzusagen, mithilfe von Softmax, Cross-Entropy und einem weiteren Gewichtsmatrix-Update:

$$p_{Token} = \frac{e^{Score_{Token}}}{\sum_k e^{Score_k}}$$

Optional: Sampling bzw. Top-k für kreative Textgeneratoren (ChatGPT etc.), statt immer den wahrscheinlichsten Token zu wählen.

Beispiel: höchste Wahrscheinlichkeit → „Frankreich“ (Token-ID 12856)



📖 Einordnung

- Gesamtüberblick verbindet: Tokenisierung → Repräsentationen → Attention → Output → Loss → Update.
- Nützlich als mentale Landkarte: Wo greifen Demos/Rechenbeispiele in die Pipeline ein?
- Vorbereitung für Modellvergleich und Bausteine des Transformers (Residuals, LayerNorm, FFN).

1.4.11 Beispiel: Softmax und Cross-Entropy (Update für W_{out})

Das Beispiel zeigt, wie das Modell durch Training an W_{out} lernt, welche Tokens es mit welcher Wahrscheinlichkeit vorhersagen soll – Ziel ist, dass das Modell lernt, welches Wort mit welchem Vektor verknüpft ist:

$$p = \text{Softmax}(z) \quad \text{Cross-Entropy Loss} = -\log(p_k) \quad \frac{\partial L}{\partial z} = p - y \quad \frac{\partial L}{\partial W} = (p - y) \cdot x^T$$

Fehler (Cross-Entropy): $L = -\log(p_{\text{Zielklasse}}) = -\log(0.6590) = 0.417$. Je wahrscheinlicher das Zielwort ist, desto kleiner ist der Fehler. In der nächsten Iteration: $p_{\text{Frankreich}} = 0.85$, $L = 0.163$ – der Fehler wurde kleiner.

Beispiel 1 (3 Klassen: Frankreich, Deutschland, Spanien): $z = (2.0, 1.0, 0.1)$, $\text{Softmax} = (0.6590, 0.2424, 0.0986)$, $y = (1, 0, 0)$.

Beispiel 2 (4 Klassen: Frankreich, Deutschland, Spanien, Italien): $z = (3.52, 2.37, 1.185, 0.07)$, $\text{Softmax} = (0.692, 0.219, 0.066, 0.022)$, $y = (1, 0, 0, 0)$.

In beiden Fällen ergibt sich die Gradiententabelle aus $p - y$ (und daraus die Gradientenmatrix $(p - y) \cdot x^T$ für alle Zeilen und Spalten).

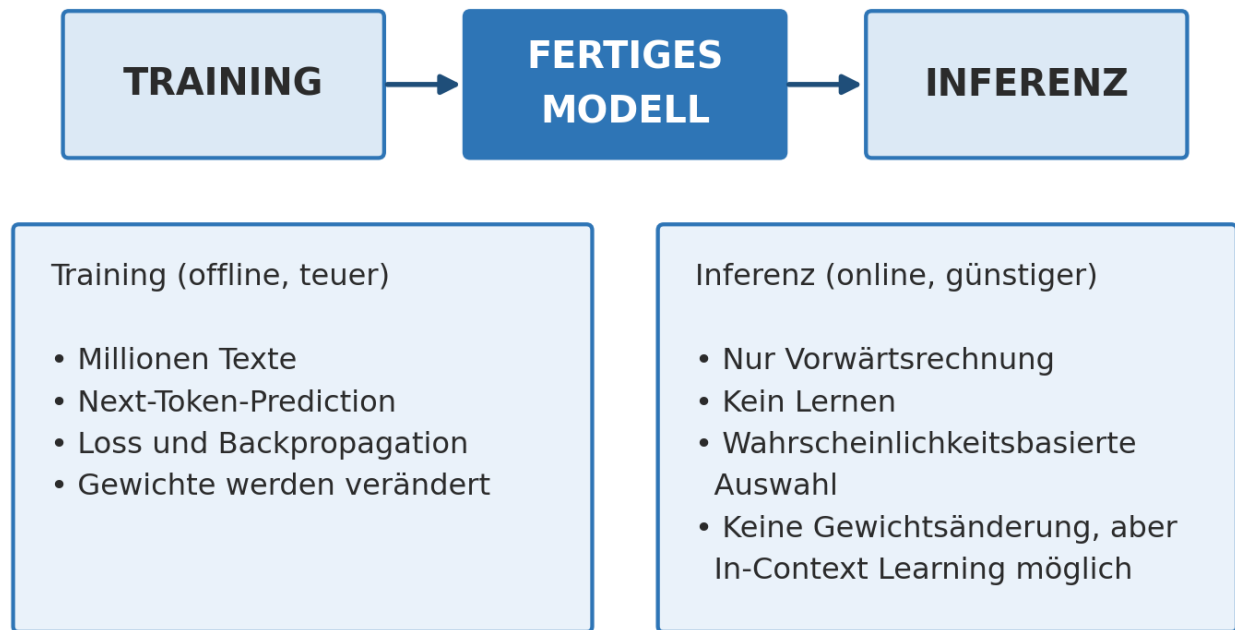
Einordnung

- Softmax & Cross-Entropy als zentrales Duo: Wahrscheinlichkeiten erzeugen und Fehler messen.
- Interpretation: Cross-Entropy bestraft „selbstsichere“ falsche Vorhersagen besonders stark.
- Verbindet mathematische Formel mit praktischem Training: Loss dient als Optimierungsziel.

Notebooks

- Softmax & Cross-Entropy (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/decoder_softmax_crossentropy_wout.ipynb): verbindet Wahrscheinlichkeitsverteilung und Fehlermass im Decoder-Output.

1.4.12 Training vs. Inferenz – Was passiert wann?



Beim Antworten lernt das Modell nicht, es rechnet nur – keine Gewichtsänderung, aber In-Context Learning ist möglich: Beispiele im Prompt können das Verhalten zur Laufzeit beeinflussen.

Bis hierhin wissen wir: Beim Antworten lernt das Modell nicht. Wenn wir wollen, dass es trotzdem aktuelles oder spezifisches Wissen nutzt, müssen wir dieses Wissen vor der Modellrechnung einspeisen. Genau das macht RAG.

Einordnung

- Klare Trennung von Training (Lernen) und Inferenz (Anwenden).
- Beim Antworten findet keine Gewichtsänderung statt – das Modell rechnet nur Wahrscheinlichkeiten.
- Aktuelles oder domänenspezifisches Wissen kann daher nicht „nachgelernt“ werden.
- Motivation für RAG: Wissen wird zur Laufzeit als Kontext ergänzt, nicht in den Gewichten gespeichert.

1.5 Systeme um das Modell: Retrieval & Kontext (RAG)

Bis hierhin wurde betrachtet, wie ein Large Language Model intern arbeitet: Tokenisierung, Embeddings, Self-Attention und Training über Wahrscheinlichkeiten.

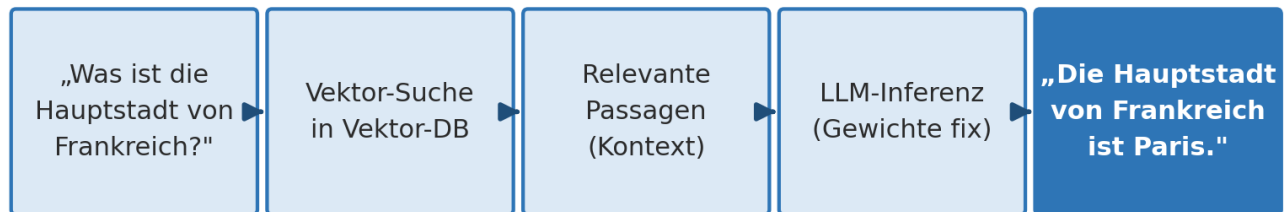
In der Praxis reicht das Modell allein jedoch selten aus. Aktuelle Anwendungen erweitern LLMs deshalb um externe Wissensquellen und Systemkomponenten, die zur Laufzeit Kontext bereitstellen.

Dieses Kapitel zeigt, wie Vektor-Datenbanken und Retrieval-Augmented Generation (RAG) das Sprachmodell zu einem wissensgestützten System erweitern – ohne das Modell selbst zu verändern.

Retrieval Augmented Generation (RAG) – Kernaussagen: Teil der Inferenz (kein Training), LLM-Gewichte bleiben unverändert, externes Wissen wird als Kontext ergänzt: Antwort = Sprachmodell + Kontext.

	Gewichte
Training	ändern sich
Inferenz	fix
RAG	Kontext wird ergänzt (Gewichte fix)
Vektor-DB	Wissensquelle

RAG-Ablauf: Frage → Retrieval → Kontext → LLM-Inferenz → Antwort



Merksatz: RAG verändert nicht das Modell - sondern den Kontext.

Einordnung

- RAG ist ein vorgeschalteter Inferenz-Schritt und kein Training.
- Die Modellgewichte bleiben unverändert; nur der Eingabekontext wird erweitert.
- Externes Wissen wird über semantische Suche (Vektor-Datenbank) bereitgestellt.
- Das Sprachmodell verarbeitet Kontext und Frage gemeinsam zur Antwortgenerierung.
- RAG erweitert Wissen zur Laufzeit, nicht durch neue Gewichte.

1.5.1 Was muss in der Vektor-Datenbank stehen?

Was muss in der Vektor-Datenbank stehen?

Keine Wörter, keine Faktenliste – Textpassagen (Chunks) mit Bedeutung.



Einordnung

- Vektor-Datenbanken speichern keine Wörter, sondern Embeddings von Textpassagen (Chunks).
- Suche erfolgt semantisch: Ähnliche Bedeutung ⇒ ähnliche Vektoren (Nearest Neighbor).
- Ergebnis ist Kontextmaterial, das ein LLM zur Antwortgenerierung nutzen kann (statt reines Parameterwissen).

Beispiel:

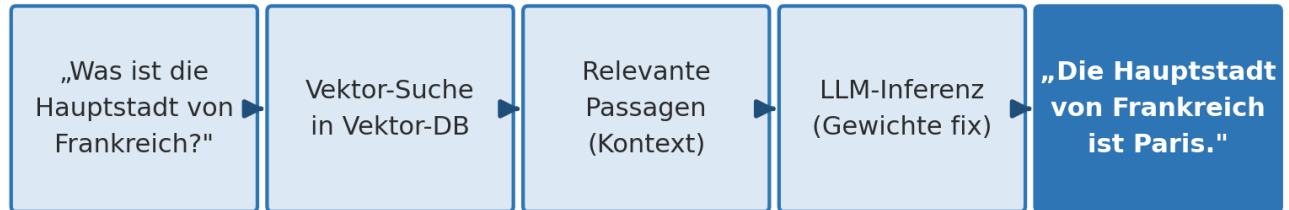
„Paris ist die Hauptstadt von Frankreich.“ Die KI sucht **nicht nach dem Wort „Paris“**, sondern nach **Texten mit gleicher Bedeutung**.

Hinweis

- Die Vektor-Datenbank ist ein semantisch durchsuchbarer Wissensindex.

1.5.2 RAG-Prinzip im Rückblick

RAG-Ablauf: Frage → Retrieval → Kontext → LLM-Inferenz → Antwort



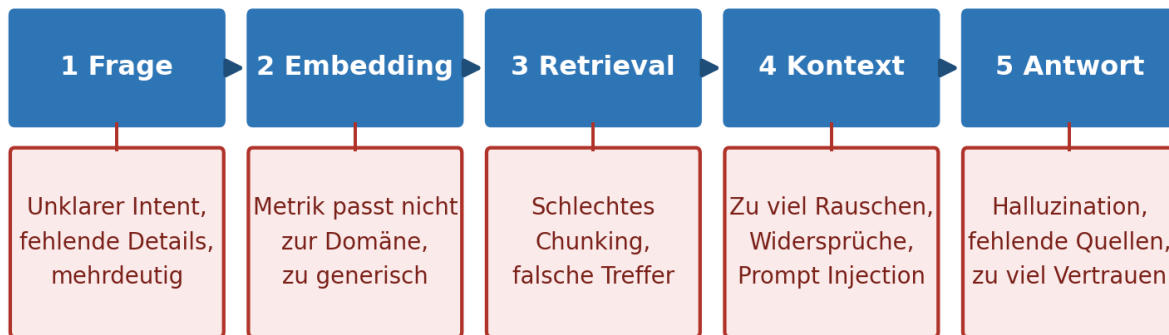
Merksatz: RAG verändert nicht das Modell - sondern den Kontext.

Einordnung

- RAG kombiniert Retrieval (Kontextbeschaffung) und Generation (Antwortformulierung).
- Ablauf: Frage → Embedding → semantische Suche → relevante Passagen → Prompt mit Kontext → LLM-Antwort.
- Vorteil: Aktualisierbares Wissen ohne Modell-Retraining; Quellen können zitiert/überprüft werden.
- Rollen: Vektor-DB als Wissensindex, LLM als Sprach- und Schlussfolgerungsmodul (auf Basis des Kontexts).

1.6 Grenzen und Fallstricke von RAG

RAG: Typische Fallstricke - wo Retrieval und Kontext scheitern können



Merksatz: Gute Antworten entstehen nur mit gutem Retrieval und sauberem Kontext.

Fehlerquellen entlang der Pipeline (Retrieval → Kontext → Antwort).

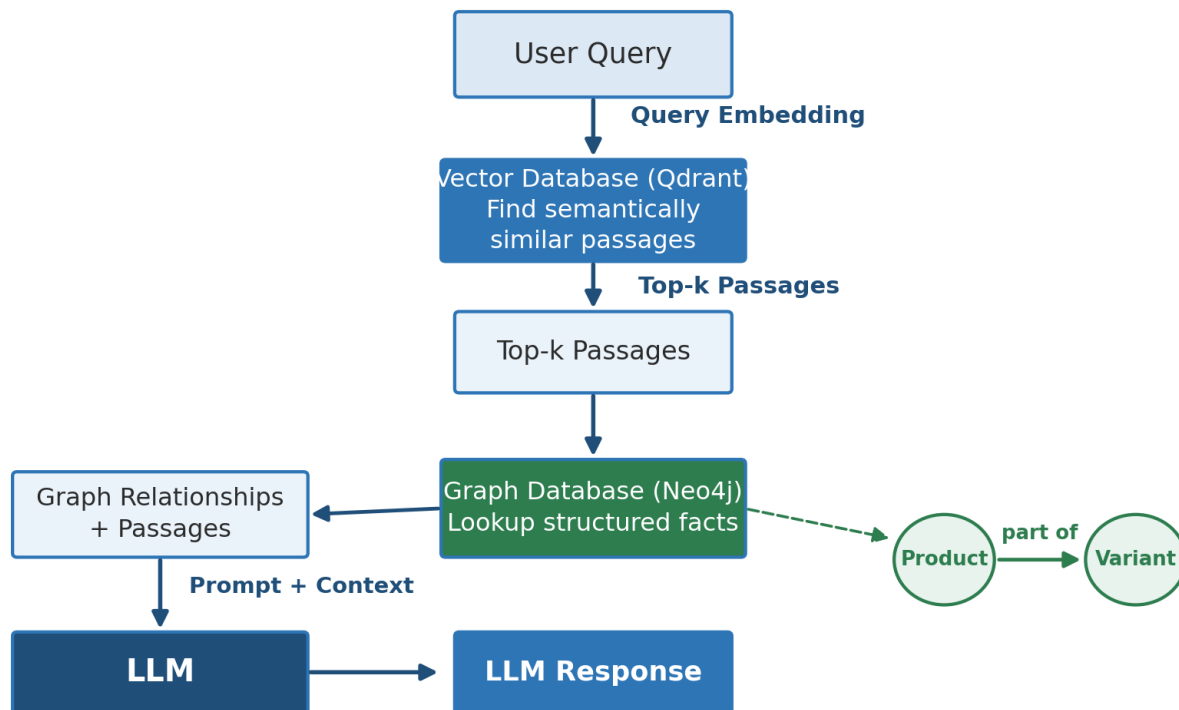
Was schiefgehen kann (und warum):

- **Schlechtes Chunking:** Zu kurze/zu lange Passagen → Kontext passt nicht zur Frage.
- **Embedding-Mismatch:** Falsches oder zu generisches Embedding-Modell → irrelevante Treffer.
- **Rauschen im Kontext:** Zu viele oder widersprüchliche Passagen verwässern die Antwort.
- **Prompt Injection:** Externe Dokumente enthalten Anweisungen, die das Modell fehlleiten.

Merksatz: RAG verbessert Aktualität und Faktentreue nur, wenn Retrieval-Qualität und Kontext-Design stimmen.

Erweiterung: Graph-RAG (Vektor-RAG + Wissensgraph):

Hybrid (Vector + Graph) RAG



Klassisches RAG nutzt eine **Vektor-Datenbank**, um semantisch ähnliche Textpassagen zu finden. Das funktioniert sehr gut – hat aber eine Grenze:

- Vektorsuche findet **ähnliche Bedeutung**,
- aber sie kennt **keine expliziten Beziehungen** wie „Produkt gehört zu Kategorie“, „Produkt hat Attribut“, „Produkt ist kompatibel mit ...“.

Graph-RAG ergänzt RAG deshalb um einen **Wissensgraphen** (z. B. Neo4j): Nach der semantischen Suche werden zusätzliche, strukturierte Kontextinformationen aus dem Graphen geladen.

Prinzip (vereinfacht): 1. Frage → Query-Embedding

2. Top-k Treffer aus der Vektor-DB (relevante Textpassagen) 3. Zu den Treffern passende Knoten/Beziehungen im Graphen abfragen (z. B. Attribute, Varianten, Kategorien) 4. **LLM erhält: semantische Passagen + strukturierte Fakten** als Kontext

Vorteile: - stabilere Faktentreue (Beziehungen sind explizit modelliert) - bessere Erklärbarkeit („Quelle“ + „Relation“) - gezielte Kontextanreicherung (weniger Zufall im Prompt)

Merksatz:

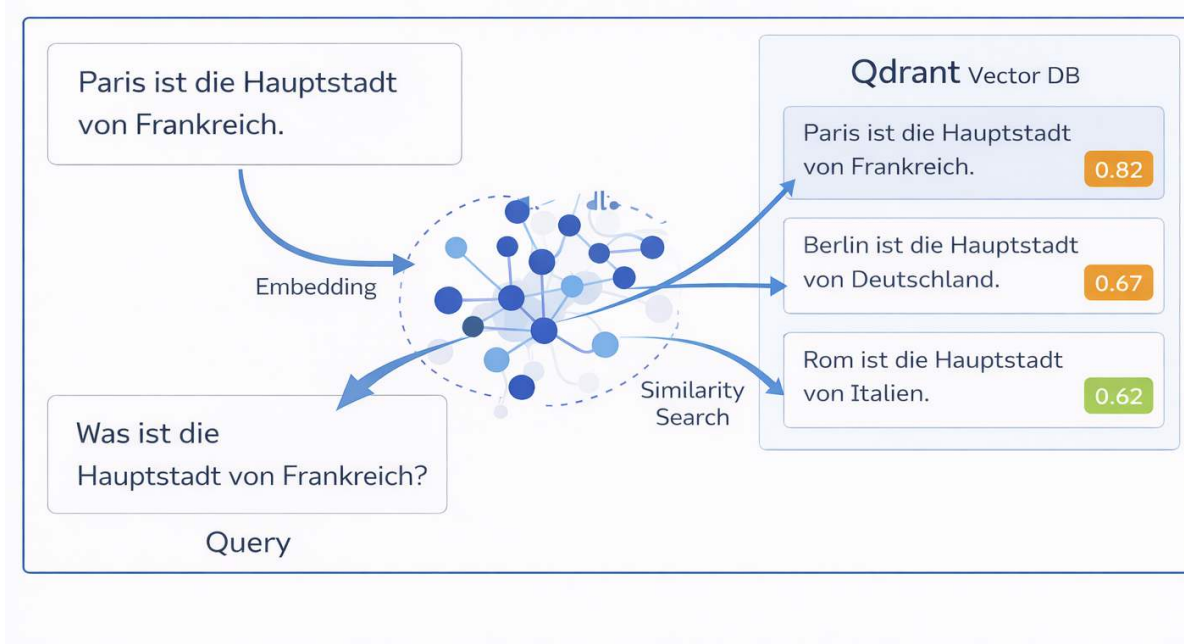
Vector-RAG liefert *ähnliche Inhalte* – Graph-RAG liefert *zusätzlich erklärbare Beziehungen*.

🚩 Hinweis

- Man kann Graph-RAG umsetzen, indem man Vektor-Treffer (Qdrant) über IDs/Metadaten mit einem Neo4j-Graphen verknüpft und daraus **zusätzlicher Kontext** extrahiert wird.

1.6.1 Live-Demo: Semantische Suche mit einer Vektor-Datenbank

Qdrant (Local Mode, ohne Docker). Der gespeicherte Text „Paris ist die Hauptstadt von Frankreich“ wird per Embedding in einen Vektor transformiert. Für die Query „Was ist die Hauptstadt von Frankreich?“ liefert die Similarity Search Ergebnisse aus der Qdrant Vector DB.



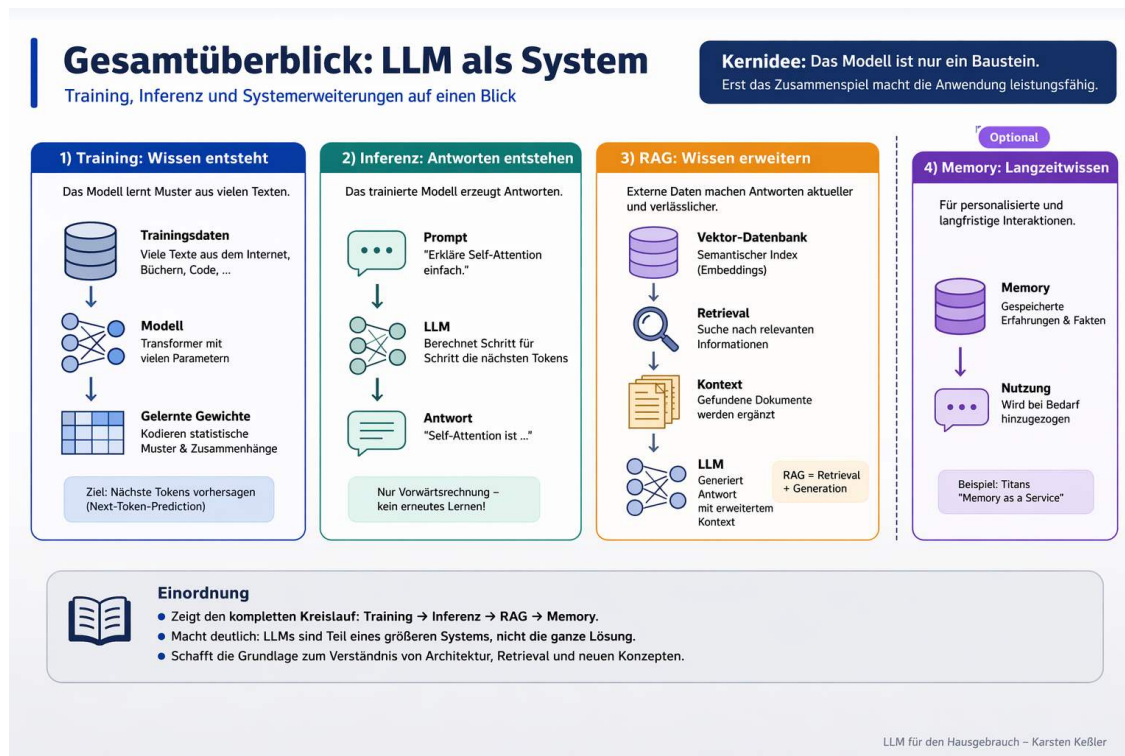
📖 Einordnung

- Demonstriert den Retrieval-Schritt von RAG in einer echten Vektor-DB (Qdrant, Local Mode).
- Pipeline: Dokumente chunking → Embeddings → Index; Anfrage → Query-Embedding → Similarity Search.
- Wichtig: Gespeichert werden Textpassagen (mit Metadaten), nicht Stichwortlisten.
- Ergebnisinterpretation: Trefferqualität hängt von Chunking-Strategie und Embedding-Modell ab.

📌 Notebooks

- RAG End-to-End Toy (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/rag_end_to_end_toy.ipynb): zeigt die komplette RAG-Kette offline von Chunks bis Antwort.
- Qdrant Demo (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/qdrant_demo.ipynb): demonstriert semantische Suche in einer echten Vektor-Datenbank.

1.7 Systemübersicht



LLM-System: Training, Inferenz und RAG – Architektur, Pipeline

Einordnung

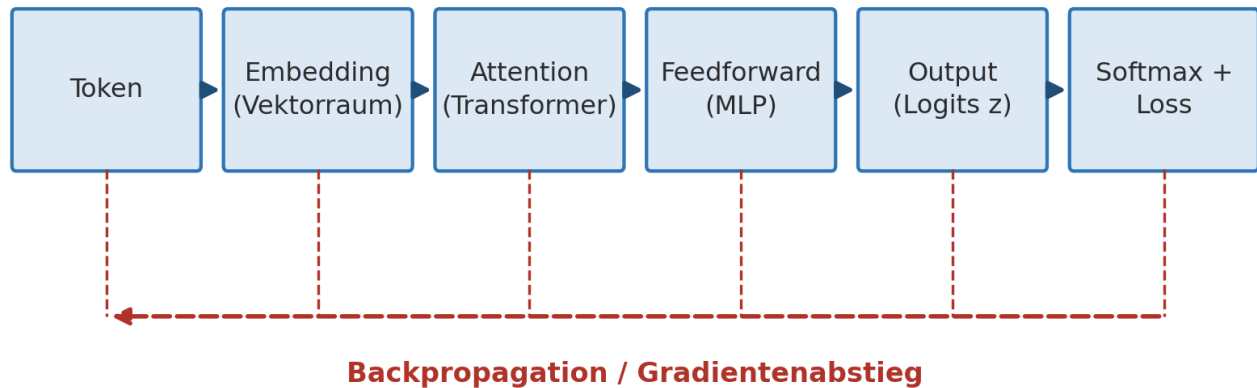
- Verbindet alle bisherigen Konzepte: Modell, Training, Inferenz und externe Systeme.
- Zeigt: LLM ist nur ein Baustein innerhalb eines größeren Systems.
- Vorbereitung für Architektur, RAG und Memory-Konzepte.

1.8 Modellarchitektur

Modellarchitektur als Einbettung der Rechenschritte: Token → Embedding (Vektorraum) → Attention (Self-Attention und Transformer-Blöcke) → Lineare Projektion ($z = W_{out} \cdot h + b$) → Softmax (p) → Loss (Cross-Entropy).

Wichtige Bestandteile:

- **Embedding-Schicht:** Token werden in Vektoren (z. B. 768 Dimensionen) umgewandelt.
- **Multi-Head Self-Attention:** Tokens schauen aufeinander, Berechnung der Kontexte.
- **Feedforward-Schichten (MLP):** Weiterverarbeitung nach Attention.
- **Lineare Projektion (Output Layer):** Berechnung der Logits (z).
- **Softmax und Cross-Entropy:** Umwandlung in Wahrscheinlichkeiten und Berechnung des Loss.
- **Backpropagation und Gradientenabstieg:** Optimierung der Gewichtsmatrizen (W_Q, W_K, W_V, W_{out}), wiederholt über viele Layer (z. B. GPT-3: 96 Transformer-Layer), trainiert auf Milliarden Beispielen.



Architekturtyp	Beispiel	Anwendung	Merkmal
Encoder-only	BERT	Klassifikation, NER	Bidirektional
Decoder-only	GPT	Textgenerierung	Autoregressiv
Encoder-Decoder	T5, BART	Übersetzung, Summarization	Encoder bidirektional, Decoder autoregressiv

Einordnung

- Vertiefung Modellarchitekturen: Bausteine wiederholen sich pro Layer (Attention + FFN + Norm + Residual).
- Skalierungsgesetze (grobe Intuition): Mehr Daten/Parameter/Compute führen oft zu besseren Modellen.
- Architekturentscheidungen beeinflussen Latenz, Speicherbedarf und Kontextlänge.

Notebooks

- Positional Encoding Beispiel (https://colab.research.google.com/github/karkessler/llm-hausgebrauch/blob/main/notebooks/positional_encoding_beispiel.ipynb): zeigt, wie Positionsinformation zu Embeddings addiert und Reihenfolge unterscheidbar wird.

1.8.1 Interpretation und Kontrolle

Softmax-Summe = 1, es werden folglich Wahrscheinlichkeitsverteilungen erzeugt. Der Gradient zeigt, wie die Gewichte geändert werden müssen, um den Fehler zu verringern. Die Jacobi-Matrix $J_{ij} = \frac{\partial p_i}{\partial z_j}$ beschreibt, wie stark kleine Änderungen der Logits z die Softmax-Ausgaben p beeinflussen (Sensitivität aller Klassen auf Logit-Änderungen). Für Softmax gilt: $J_{ij} = p_i(\delta_{ij} - p_j)$.

LLMs sind Wahrscheinlichkeits-Vervollständiger: Sie schätzen Wahrscheinlichkeiten für sinnvolle nächste Tokens (Vervollständigung von Texten).

	Definition	Kontext	Beispiel
Gradient	Vektor der partiellen Ableitungen einer skalaren Funktion	für Funktionen $L(z)$, z. B. den Loss	∇L
Jacobi-Matrix	Matrix aller partiellen Ableitungen einer vektorwertigen Funktion	wenn eine Funktion mehrere Ausgaben hat, z. B. Softmax $p = f(z)$	J

Gradient = Richtung der Fehlerkorrektur. **Jacobian** = Sensitivität der Ausgaben (p) auf Änderungen der Logits (z).

Einordnung

- Zusammenfassung der Architektur-Bausteine und ihrer Rollen (Aufmerksamkeit, Nichtlinearität, Stabilisierung).
- Mentales Modell: Attention mischt Information über Positionen, FFN transformiert pro Position.
- Brücke zu neueren Varianten: effiziente Attention, längere Kontexte, Speichermechanismen.

1.8.2 Biologisches Neuron vs. Künstliches Neuronales Netz

Biologisches Neuron	Künstliches Neuronales Netz
Neuronen (Zellen)	Vektoren
Synapsen (Verbindungen, Verstärkung/Abschwächung)	Gewichtsmatrizen (z. B. W_Q, W_K, W_V)
Signalweitergabe per Neurotransmitter	Matrix-Multiplikation und Aktivierungsfunktionen (Attention, GeLU im Feedforward)
Lernen durch Plastizität (z. B. synaptische Verstärkung)	Gradientenabstieg (Backpropagation)
Sehr energieeffizient, biologisch flexibel	Hoher Rechenaufwand, hochparallel auf GPUs

Was ist die Mathematik dahinter? Vektoren, Matrizen und Wahrscheinlichkeiten sind die Grundbausteine. Matrixmultiplikation und Aktivierungsfunktionen erzeugen „Verstehen“, Softmax erzeugt Wahrscheinlichkeiten, und Cross-Entropy misst, wie falsch das Modell liegt – das macht Lernen durch Backpropagation möglich.

Aktivierungsfunktionen im Vergleich. GeLU ist die im Feedforward-Block gängige Aktivierungsfunktion moderner Transformer, aber zwei einfachere Klassiker helfen beim Verständnis:

- **Sigmoid:** $\sigma(x) = \frac{1}{1 + e^{-x}}$ – quetscht jeden Wert auf $(0, 1)$; klassisch für binäre Klassifikation und die Basis von Gating-Mechanismen (z. B. bei Titans, Folie zu Architekturvergleich), die entscheiden, wie viel Information durchgelassen wird.
- **ReLU:** $\text{ReLU}(x) = \max(0, x)$ – die einfachste und rechnerisch günstigste Nichtlinearität, historischer Vorgänger von GeLU im Feedforward-Block; GeLU ist im Kern eine glatte, differenzierbare Version von ReLU.

Einordnung

- Gesamtüberblick der Modellarchitekturen: Einordnung verschiedener Familien und Trade-offs.
- Wichtige Achsen: Kontextlänge, Compute pro Token, Trainingsobjective, Einsatzgebiet.
- Vorbereitung für den Ausblick auf Titans/Memory-Konzepte.

1.8.3 Was fehlt in dem Rechenbeispiel?

Baustein	Rechenbeispiel	GPT/BERT
Tokenization	ja	ja
Embedding	ja	ja
Self-Attention	ja	ja
Feedforward-Block (MLP mit GeLU)	nein (weggelassen für Einfachheit)	ja
Residual Connections	nein	ja
LayerNorm	nein	ja

Einordnung

- Im Rechenbeispiel bewusst vereinfacht: Feedforward-Block (MLP), Residual Connections und LayerNorm fehlen.
- Feedforward (MLP): verarbeitet jedes Token-Embedding zusätzlich einzeln, erhöht Ausdruckskraft über reine Attention hinaus.
- Residual Connections: leiten die Eingabe zusätzlich unverändert weiter und stabilisieren das Training tiefer Netze.
- LayerNorm: normiert Zwischenergebnisse und verhindert instabile Wertebereiche über viele Schichten.

1.8.4 Was im Rechenbeispiel vereinfacht wurde

Baustein	Rechenbeispiel	GPT/BERT
Tokenisierung	ja	ja
Embedding	ja	ja
Self-Attention	ja	ja
Feedforward (MLP)	nein (weggelassen, z. B. GeLU-Aktivierung)	ja
Residuals	nein	ja
LayerNorm	nein	ja

Warum diese Vereinfachung? Fokus auf das Kernprinzip Attention + Gradienten – Komplexität reduziert für besseres Verständnis.

Einordnung

- Reale Modelle (GPT/BERT) wiederholen Attention + FFN + Residual + LayerNorm über viele Layer (z. B. 12 bis 96).
- Die Vereinfachung im Rechenbeispiel diente allein der Nachvollziehbarkeit von Q/K/V, Scores und Gradienten.
- Übergang: Die nächsten Folien ordnen diese Bausteine in vollständige Modellarchitekturen ein.

1.8.5 Wo arbeiten LLMs mit Tensoren?

Objekt	Tensor-Rang	Beispiel
Eingabevektor (Token)	1	(1, 2, 1, 0)
Gewichtsmatrizen (W_Q, W_K, W_V)	2	4×4
Attention-Scores	1	Score-Liste
Attention-Output-Vektor	1	(0.304, 0.529, 0.603, 0.399)
Multi-Head Attention (echte LLMs)	4	(Batch, Heads, Seq, Seq)
Komplette Modell-Batches	3	(Batch, Sequence, Hidden), z. B. (32, 128, 768)

Einordnung

- Reale LLMs rechnen nicht mit einzelnen Vektoren, sondern mit Tensoren höherer Ordnung (Batch, Sequenz, Hidden-Size, Heads).
- Das Rechenbeispiel nutzt Rang-1/Rang-2-Objekte (Vektor, Matrix); GPT/BERT verarbeiten Rang-3/4-Tensoren parallel.
- Praktische Konsequenz: Parallelisierung auf GPUs/TPUs ist nur durch diese Tensor-Batch-Struktur effizient möglich.

1.9 Ausblick: Titans

Titans (Modellarchitektur): Weiterentwicklung klassischer Transformer-Architekturen – alternative Blockdesigns (Attention + MLP + Gating), verbesserte Positionembeddings (Rotary, State-Space Embeddings), bessere Parallelisierung und längere Kontexte. Forschungsansatz von Google DeepMind (2025).

TPU v5p (Hardware): Spezialhardware für LLM-Training, optimiert für massive Matrixoperationen, tausende TPUs pro Cluster. Das Grundprinzip bleibt dabei gleich: Tensoroperationen, Softmax, Cross-Entropy, Gradientenabstieg.

Titans erweitern dieses Grundprinzip um ein Memory-Konzept:

- **Surprise** sagt dem System, dass etwas neu oder unerwartet war – die Überraschung wird gespeichert (Surprise Memory).
- **Long-Term Memory** speichert den Überraschungszustand länger, wenn er häufig auftritt.
- **Adaptive Forgetting** entfernt alte oder irrelevante Informationen automatisch.
- **Momentum** sorgt dafür, dass das Memory weich und stabil angepasst wird.
- **Persistent Memory** speichert dauerhaft aufgabenübergreifendes Wissen.

Einordnung

- Hinweis: Titans ist ein aktueller Ansatz. Die Details können sich noch ändern.
- Zusammenfassung Titans: Memory als dritte Säule neben Parametern und Prompt-Kontext.
- Einordnung: unterschiedliche Speicherarten lösen unterschiedliche Probleme (Kontextlänge vs. Langzeitkonsistenz).
- Ausblick: Kombinationen aus RAG, Caching und Memory-Architekturen in modernen Systemen.

1.10 Evaluation: Antworten prüfen

Ein kurzer, robuster Checksatz mit vier Kriterien: **Grounding** (stehen die Aussagen im bereitgestellten Kontext?), **Quellen** (sind Zitate/Referenzen nachvollziehbar?), **Abdeckung** (wurden alle Teilfragen beantwortet?) und **Robustheit** (bleibt die Antwort bei kleinen Variationen stabil?).

Minimaler Prüfpfad für LLM-Antworten:

- **Grounding:** Stehen die Aussagen im bereitgestellten Kontext?
- **Quellen:** Sind Zitate/Referenzen nachvollziehbar und relevant?
- **Abdeckung:** Wurden alle Teilfragen beantwortet?
- **Robustheit:** Konsistente Antwort bei leicht variierten Prompts?
- **Unit-Tests:** Für strukturierte Ausgaben feste Prüfkriterien definieren.
- **Smoke Prompts:** Kurze Standardfragen für schnelle Qualitätschecks.

Merksatz: Gute Antworten sind prüfbar – nicht nur plausibel.

1.10.1 Titans – Architekturvergleich

Komponente	Rechenbeispiel	GPT/BERT	Titans
Tokenization	ja	ja	ja
Embedding	ja	ja	ja (SSM State Space Position Encoding)
Self-Attention	ja	ja	ja (verbessert: multi-query, Gating, SSM)
Feedforward (MLP)	nein	ja (GeLU)	ja (Gating)
Residuals/LayerNorm	nein	ja	ja
Gating	nein	nein	ja
Position Embedding	implizit	ja (rotary)	ja (Rotary, SSM)

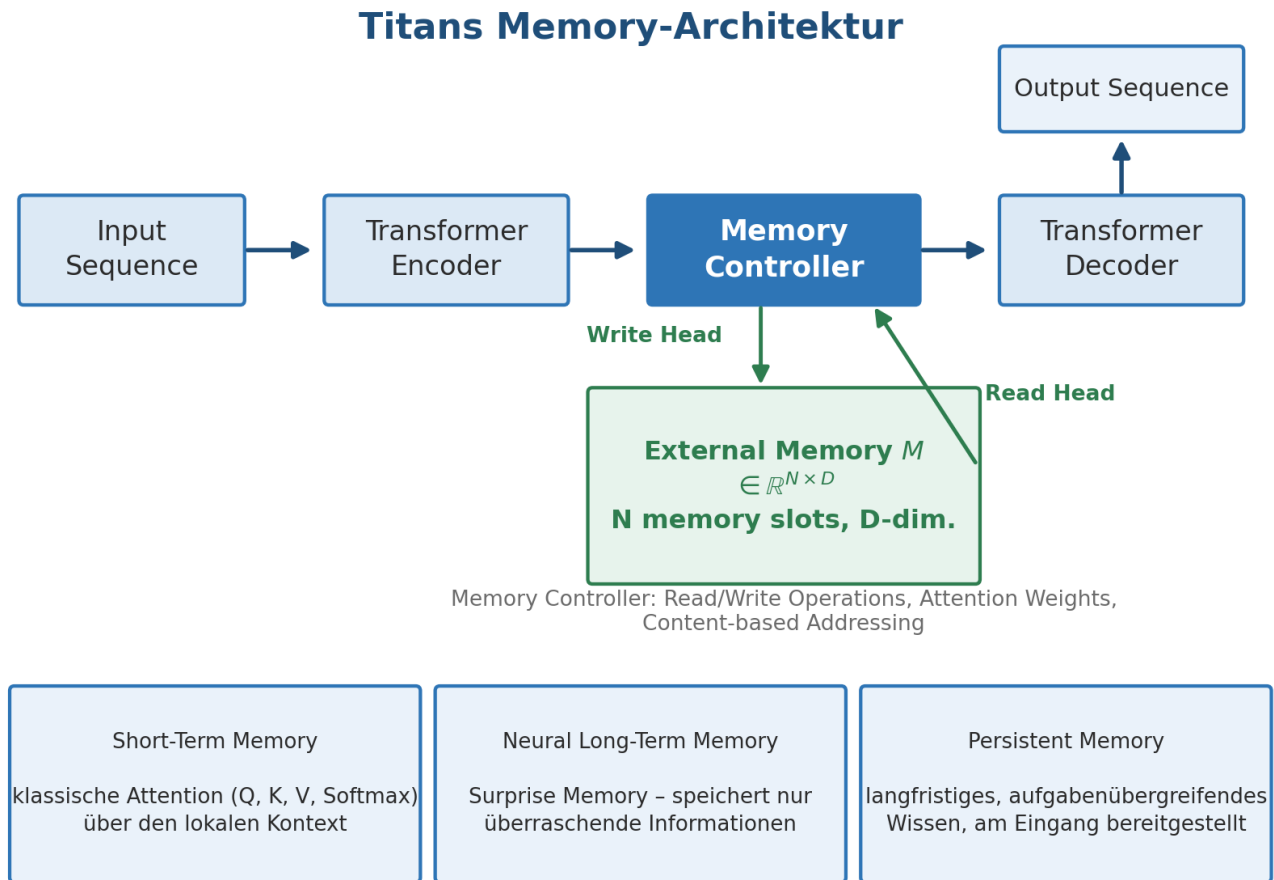
Gating funktioniert wie ein neuronaler Schalter, der entscheidet, ob Informationen im Netzwerk weitergegeben oder unterdrückt werden. **MLP** (Multilayer Perceptron) bezeichnet vollständig verbundene neuronale Netze, die nach der Attention-Schicht folgen – in GPT kommt hier z. B. GeLU zum Einsatz. Titans verwenden zusätzlich ein Gating, also eine Art Filtermechanismus.

Einordnung

- Titans erweitert die klassischen Transformer-Bausteine (Attention, FFN, Residual/LayerNorm) um einen Gating-Mechanismus.
- Gating steuert, welche Informationen ins Long-Term Memory geschrieben oder daraus gelesen werden.
- Positional Encoding wird bei Titans zusätzlich um State-Space-Elemente (SSM) ergänzt, nicht nur um Rotary Embeddings.

1.10.2 Titans Memory-Architektur

Titans erweitert klassische Transformer um Memory-Komponenten: Die neue Gedächtnisarchitektur kombiniert klassische Attention (Short-Term Memory) mit zusätzlichen Speichersystemen (Neural Long-Term Memory, Persistent Memory) und einem Adaptive-Forgetting-Gating, das gezieltes Vergessen steuert.



Warum Titans? Skalierbar auf extrem lange Kontexte (über 2 Mio. Tokens), stabileres und selektiveres Lernen.

Hauptmerkmale: Differenzierbares externes Gedächtnis, Content-based Addressing mit Attention, getrennte Read/Write-Operationen.

Vorteile: Bessere Langzeit-Abhängigkeiten, skalierbare Gedächtniskapazität, End-to-End trainierbar.

Einordnung

- Short-Term Memory (klassische Attention) verarbeitet lokalen Kontext; Long-Term Memory speichert nur „überraschende“ Informationen.
- Ziel: relevante Informationen selektiv behalten (komprimieren) statt alles im Kontextfenster mitzuführen.
- Parallele zu RAG: Auch dort wird Wissen extern gehalten, allerdings über eine separate Vektor-Datenbank statt gelernter Memory-Module.

1.11 Grenzen des klassischen Language Modells

- **Exposure Bias:** Das Modell wird auf korrekten Trainingsdaten trainiert (Ground Truth), generiert aber zur Inferenzzeit Token für Token autoregressiv – kleine Fehler im frühen Output können sich akkumulieren und zu zunehmend abweichenden Antworten führen.

Diese bekannten Schwächen gelten auch für GPT-Modelle wie ChatGPT:

- **Halluzinationen:** Generierung von falschen Inhalten mit hoher Überzeugungskraft – das Modell verteidigt von sich aus überzeugt die eigenen Fehler.
- **Wissensgrenze:** Modelle kennen keine Ereignisse nach dem Trainingszeitpunkt.
- **Fehlende Quellenangaben:** Keine direkten oder überprüfbaren Zitate verfügbar.
- **Datenschutz und Zugriff:** Kein direkter Zugriff auf externe/private Datenquellen (ohne Tools/RAG) – das Modell sieht nur den Prompt-Kontext.
- **Begrenzte Kontextlänge:** Nur eine begrenzte Anzahl an Tokens kann gleichzeitig verarbeitet werden.
- **Vergessen von Informationen:** Das Modell hat Probleme, sich über lange Zeiträume Informationen zu merken.
- **Kosten:** Hohe Kosten für das Generieren langer Texte – die quadratische Abhängigkeit von Rechenzeit und Speicher von der Sequenzlänge ist der Hauptgrund für die Begrenzung der Kontextlänge.

GPT versteht überhaupt nichts, es kann nur sehr gut Wahrscheinlichkeiten modellieren. GPT hat keine Gefühle oder Intentionen, es berechnet nur die wahrscheinlichste Wortfolge. GPT kann scheitern bei: Logikrätseln, Rechenaufgaben mit langen Ketten, Fakten nach dem Trainingszeitpunkt.

Einordnung

- Grenzen von LLMs: Halluzinationen, begrenztes Kontextfenster, Kosten (Compute/Token).
- Sicherheits- und Qualitätsaspekte: Prompt Injection, Datenleaks, Bias und fehlende Garantien.
- Konsequenz für Praxis: Evaluierung, Quellen, Guardrails und Human-in-the-loop bei kritischen Anwendungen.

1.12 Fazit und Diskussion

Dieses Dokument hat gezeigt, dass Large Language Models keine „magischen Systeme“ sind, sondern auf klar nachvollziehbaren mathematischen Prinzipien basieren:

- **Wahrscheinlichkeiten statt Wissen**
- **Self-Attention als Kernmechanismus**
- **Training vs. Inferenz klar getrennt**

Gleichzeitig zeigt sich:

- Ohne Kontext (z. B. durch RAG) sind Modelle **wissensbegrenzt**
- Ohne Prüfung sind Antworten **nicht verlässlich**
- In Kombination mit Systemen entstehen **sehr leistungsfähige Anwendungen**

Ein erkennbarer struktureller Trend führt dabei von einzelnen Modellen hin zu Systemen aus mehreren zusammenarbeitenden LLM-Instanzen (Multi-Agenten-Systemen), die Teilaufgaben untereinander aufteilen und koordinieren. Unabhängig davon, wie viele Modelle zusammenwirken, bleiben Tokenisierung, Attention und die Trennung von Training und Inferenz die gemeinsame Grundlage – wer diese Mechanik versteht, versteht auch komplexere Systeme, die darauf aufbauen.

Diskussionsfragen:

- Reicht statistisches Lernen für „Verstehen“ aus?
- Wie viel Kontrolle brauchen wir über generative Systeme?
- Welche Rolle spielen LLMs in zukünftigen Softwarearchitekturen?

1.13 Glossar

Begriff	Kurzerklärung	Verwendet in
Adaptive Forgetting	Zielgerichtetes Vergessen irrelevanter Informationen	Titans Memory
Äußeres Produkt	Produkt aus Spaltenvektor mal Zeilenvektor: ergibt eine Matrix (Rang 1), wichtig für Gradienten wie $\partial L / \partial W$ (z. B. $(p - y) \cdot x^T$). Nicht zu verwechseln mit dem Kreuzprodukt (nur in 3D definiert, liefert einen Vektor)	Gradientberechnung, Matrixformeln
Backpropagation	Rückrechnung des Fehlers zur Gewichtoptimierung	Gradient-Updates
Cross-Entropy Loss	Berechnet den Fehler der Vorhersage	Training
Decoder-only	Modelltyp, der nur Vorwärtsrichtung nutzt (z. B. GPT)	Textgenerierung, GPT-Architektur, Vorhersage des nächsten Tokens
Embedding	Umwandlung von Tokens in Vektoren	Eingabeschicht
Encoder-Decoder	Zwei-Komponenten-Modell für z. B. Übersetzungen (z. B. T5, BART)	Übersetzung, Zusammenfassung, Aufgaben mit getrennter Eingabe und Ausgabe
Encoder-only	Modelltyp mit vollem Kontexteinblick (z. B. BERT)	Klassifikation, Masked Language Modeling (z. B. BERT), Eingabeanalyse
Feedforward (MLP)	Weiterverarbeitung pro Token nach Attention	GPT/BERT Architektur
Gating	Mechanismus zur selektiven Signalweitergabe	Titans Feedforward
Gradientenabstieg	Algorithmus zur schrittweisen Fehlerreduktion	Training, Update-Regel
Head	Ein einzelner Aufmerksamkeitsmechanismus, der Query, Key und Value verarbeitet	Bestandteil von Multi-Head Attention
Jacobi-Matrix	Sensitivität der Ausgaben auf die Eingaben	(kurz bei Softmax-Ableitung)
LayerNorm	Normierung der Zwischenausgaben	GPT/BERT Architektur

Begriff	Kurzerklärung	Verwendet in
Likelihood	Wahrscheinlichkeit für die Vorhersage des korrekten nächsten Tokens gegeben den bisherigen Kontext. Das Modell maximiert diese Wahrscheinlichkeit im Training	Cross-Entropy, Training, Softmax-Ausgabe
Logits (z)	Rohwerte vor der Softmax-Normalisierung	Vor Softmax
Matrix	2-dimensionales Zahlenarray (z. B. Gewichts- oder Embedding-Matrizen)	Gewichtsmatrizen, Attention, Embedding
Mixture of Experts (MoE)	Dynamische Auswahl von Modellkomponenten	(nicht im Vortrag, nur am Rande in Titans möglich)
Multi-Head Attention	Parallele Anwendung mehrerer Heads zur besseren Erfassung unterschiedlicher Beziehungsmuster	Attention-Formeln, Erweiterung in LLMs: Titans Architektur
Neural Long-Term Memory	Speicherung überraschender Informationen	Titans Memory
Persistent Memory	Langfristige Aufgabenrepräsentation	Titans Memory
Positional Encoding	Positionsinformation für Reihenfolge der Tokens	Eingabeschicht Attention
p-y (Gradient)	Fehlermaß zwischen Vorhersage und Ziel	Gradient, Backpropagation
Q, K, V (Query, Key, Value)	Mathematische Projektionen, die Attention ermöglichen	Attention-Formeln
Query (Q)	Repräsentation des aktuellen Tokens, das Informationen aus anderen Tokens anfordert („Wen schaue ich an?“)	Attention-Formeln, Frage
Key (K)	Repräsentation aller Tokens, die betrachtet werden können („Wer bietet sich an?“)	Attention-Formeln, Angebot
Value (V)	Inhaltliche Informationen der Tokens, die (gewichtete) Antworten liefern („Welche Information bekomme ich?“)	Attention-Formeln, Information
RAG (Retrieval-Augmented Generation)	Kombination aus Sprachmodell (LLM) und externem Datenabruf: Vor der Antwort werden passende Dokumente oder Fakten aus z. B. Datenbanken oder Wissensquellen geholt und als Kontext in das Modell eingebettet	Kontextverarbeitung, Anwendung in Agentforce, Verbindung von LLM und externem Wissen
Residual Connection	Hinzufügen der Eingangswerte zur Stabilisierung	GPT/BERT Architektur
Schwellenwert	Wert, ab dem eine Entscheidung getroffen wird (z. B. Aktivierungsfunktion, binäre Klassifikation)	Neuronen, Aktivierung, Klassifikation
Self-Attention	Tokens beziehen gegenseitig Informationen aufeinander	Hauptblock Attention
Skalarprodukt	Multiplikation zweier Vektoren mit Ausgabe eines Skalars (z. B. Ähnlichkeitsmaß)	Attention-Berechnung, Ähnlichkeit
Softmax	Umwandlung von Scores in Wahrscheinlichkeiten	Attention, Output Layer
SSM (State Space Model)	Alternative für Positional Encoding in Titans	Titans Position-Encoding
Surprise Memory	Memory-Komponente, die seltene oder überraschende Muster speichert	Titans-Architektur
Tensor	Mehrdimensionale Verallgemeinerung von Vektoren und Matrizen	Eingabedaten, Modellparameter, Attention-Berechnungen
Titans Architektur	Erweiterte Transformer-Architektur mit Gating & Memory	Ausblick
Token	Kleinste Einheit des Inputs (z. B. Wort, Subwort, Zeichen)	Tokenisierung, Eingabe
Tokenization	Zerlegung des Textes in Tokens	Vorverarbeitung
TPU (Tensor Processing Unit)	Spezialisierte Hardware für LLM-Training	Titans Hardware
Vektor	Zahlenliste zur Darstellung von Tokens	Embedding, Attention

1.14 Literaturverzeichnis

Grundlegende Architektur

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30 (NeurIPS 2017), S. 5998–6008. <https://arxiv.org/abs/1706.03762>

Tokenisierung und Embeddings

Sennrich, R., Haddow, B. & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016), S. 1715–1725. <https://arxiv.org/abs/1508.07909>

Mikolov, T., Chen, K., Corrado, G. & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. Proceedings of the Workshop at ICLR 2013. <https://arxiv.org/abs/1301.3781>

GPT-Modelle

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Technical Report. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems 33 (NeurIPS 2020), S. 1877–1901. <https://arxiv.org/abs/2005.14165>

Retrieval-Augmented Generation

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems 33 (NeurIPS 2020), S. 9459–9474. <https://arxiv.org/abs/2005.11401>

Neuere Architekturen

Behrouz, A., Zhong, P. & Mirrokni, V. (2025). *Titans: Learning to Memorize at Test Time*. Google DeepMind. arXiv:2501.00663. <https://arxiv.org/abs/2501.00663>

Grundlagen Deep Learning

Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>

Lehrbuch (LLM-Praxis)

Alammar, J. & Grootendorst, M. (2024). *Hands-On Large Language Models*. O'Reilly Media. ISBN 978-1-098-15079-0.

Begleitmaterial

Keßler, K. (2025). *LLM für den Hausgebrauch – Notebooks und Materialien*. GitHub: <https://github.com/karkessler/llm-hausgebrauch>
Web: <https://tutor.kkessler.de/llm>
DOI: <https://doi.org/10.5281/zenodo.18293327>